

Recuadro de aplicación

Sears, Roebuck and Company (comúnmente conocida con el nombre de **Sears**), fundada en 1886, creció hasta convertirse en la más grande tienda minorista multilínea en Estados Unidos a mediados del siglo xx. En la actualidad es uno de los minoristas más grandes en el mundo en la venta de mercancías y servicios. Asimismo, cuenta con el servicio más grande de entrega a domicilio de muebles y electrodomésticos de Estados Unidos, donde realiza más de 4 millones de entregas al año. En este país, esta tienda controla una flota de más de 1 000 vehículos para entrega de mercancía, en la cual se incluyen vehículos contratados y propios. Además cuenta, en Estados Unidos, con una flota de alrededor de 12 500 vehículos de servicio y los técnicos correspondientes, quienes realizan alrededor de 15 millones de llamadas de servicio a domicilio al año para reparar e instalar electrodomésticos, así como para brindar servicios de mejoras en el hogar.

El costo asociado con la operación de este enorme sistema de entrega a domicilio y servicio a los hogares se encuentra en el orden de *miles de millones de dólares al año*. Debido a los miles de vehículos que se utilizan para atender a las solicitudes de decenas de miles de llamadas de clientes *diariamente*, la eficiencia de esta operación tiene un efecto sorprendente en la rentabilidad de la compañía.

Para poder atender tantas llamadas de servicio por parte de los clientes con tantos vehículos, *cada día* es necesario tomar un enorme número de decisiones. ¿Qué paradas se le debe asignar a cada ruta? ¿Cuál debe ser el orden de dichas paradas (lo cual tiene un efecto significativo en la distancia y tiempo totales de la ruta) de cada vehículo? ¿Cómo pueden

tomarse todas estas decisiones de tal manera que se minimicen los costos totales de operación y a la vez se ofrezca un servicio satisfactorio a los clientes?

Fue evidente que era necesario que la investigación de operaciones considerara este problema. La formulación natural del problema debía asumir la forma de un *problema de enrutamiento de vehículos con ventanas de tiempo* (VRPTW, vehicle-routing problem with time windows), para el cual se han desarrollado algoritmos exactos y heurísticos. Desafortunadamente, el problema de Sears es tan grande que representa un problema de optimización combinatoria muy difícil que está más allá del alcance de los algoritmos estándar de VRPTW. Por esta razón se desarrolló un nuevo algoritmo basado en el uso de la *búsqueda tabú* con el fin de tomar decisiones acerca de qué ruta vehicular atenderá qué paradas y cuál será la secuencia de paradas dentro de una cierta ruta.

El nuevo sistema de enrutamiento y programación de vehículos, el cual se basó en gran medida en la búsqueda tabú, representó para Sears un **ahorro por evento de más de 9 millones de dólares** y **ahorros anuales de más de 42 millones de dólares**. Asimismo, brindó un gran número de beneficios intangibles en los que se incluye un *mejor servicio* al cliente.

Fuente: D. Weigel y B. Cao: “Applying GIS and OR Techniques to Solve Sears Technician-Dispatching and Home-Delivery Problems”, en *Interfaces*, **29**(1): 112-130, enero-febrero de 1999. (En el sitio en internet de este libro —www.mhhe.com/hillier— se proporciona una liga hacia este artículo.)

temporal los movimientos que pudieran regresar el proceso a una solución visitada recientemente. Una **lista tabú** registra estos movimientos prohibidos, los cuales se conocen como *movimientos tabú*. (La única excepción a la prohibición de un movimiento de este tipo se presenta cuando un movimiento tabú en realidad es mejor que la mejor solución factible que se haya encontrado hasta ese momento.)

Este uso de *memoria* para dirigir la búsqueda al utilizar listas que registran cierta parte de la historia reciente es una característica distintiva de la búsqueda tabú. Esta condición tiene raíces en el campo de la inteligencia artificial.

La búsqueda tabú también puede incorporar algunos conceptos avanzados. Uno de ellos es la *intensificación*, que implica la exploración de una parte de la región factible con más intensidad de la usual después de que se haya identificado como una parte particularmente promisoría para encontrar en ella muy buenas soluciones. Otro concepto es la *diversificación*, que implica forzar la búsqueda por medio de la entrada en áreas de la región factible que no se han explorado con anterioridad. (Para implantar ambos conceptos se utiliza la memoria de largo plazo.) Sin embargo, el enfoque aquí se centrará sobre la forma básica de la búsqueda tabú que se resume más adelante sin ahondar en estos conceptos adicionales.

Esquema de un algoritmo de búsqueda tabú básico

Paso inicial. Comience con una solución de prueba inicial factible.

Iteración. Utilice un procedimiento de búsqueda local apropiado para definir los movimientos factibles en la vecindad local de la solución de prueba actual. No considere la realización de ningún movimiento incluido en la lista tabú actual a menos que ese movimiento genere una mejor solución que la mejor solución de prueba que se haya encontrado hasta ahora. Determine cuál de los movimientos restantes proporciona la mejor solución. Adopte esta solución como la próxima solución

de prueba, sin importar si ésta es mejor o peor que la solución de prueba actual. Actualice la lista tabú para evitar el regreso a la última solución de prueba actual. Si la lista tabú ya está llena, elimine el elemento más antiguo de la lista para proporcionar más flexibilidad a los movimientos futuros.

Regla de detención. Utilice algún criterio de detención, como un número fijo de iteraciones, una cantidad fija de tiempo del CPU o un número fijo de iteraciones consecutivas que no produzcan ninguna mejora al mejor valor de la función objetivo. (El último es un criterio particularmente popular.) El proceso también puede detenerse en cualquier iteración donde no existan movimientos factibles en la vecindad local de la solución de prueba actual. Se acepta la mejor solución de prueba que se haya encontrado en cualquier iteración como la solución final.

Este esquema deja algunas preguntas sin contestar.

1. ¿Cuál procedimiento de búsqueda local debe usarse?
2. ¿Cómo debe definir ese procedimiento la *estructura de vecindad* que especifica cuáles soluciones son vecinas inmediatas de cualquier solución de prueba (que se pueden buscar en una misma iteración)?
3. ¿Cuál es la forma en la que los movimientos tabú deben representarse en la lista tabú?
4. ¿Cuál movimiento tabú debe agregarse a la lista tabú en cada iteración?
5. ¿Qué tanto debe mantenerse un movimiento tabú en la lista?
6. ¿Cuál regla de detención debe usarse?

Todos éstos son detalles importantes que deben trabajarse para satisfacer el tipo específico de problema que se aborde, como lo ilustran los siguientes ejemplos. La búsqueda tabú sólo proporciona una estructura general y directrices estratégicas para desarrollar un método heurístico específico para afrontar una situación particular. La selección de estos parámetros es una parte clave en el desarrollo de un método heurístico exitoso.

Los siguientes ejemplos ilustran el uso de la búsqueda tabú.

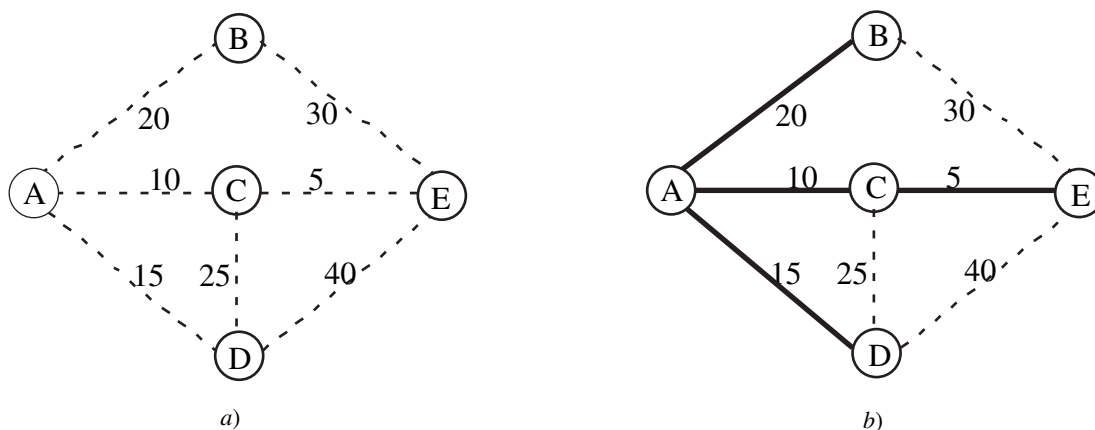
Problema del árbol de expansión mínima con restricciones

En la sección 9.4 se describe el problema del árbol de expansión mínima. En resumen, éste se inicia con una red que tiene sus nodos pero todavía no las ligaduras entre ellos, por lo que el problema es determinar cuáles ligaduras deben insertarse en la red. El objetivo es minimizar el costo (o la longitud) total de las ligaduras insertadas que proporcionarán una ruta entre cada par de nodos. En el caso de una red con n nodos se necesitan $(n - 1)$ ligaduras (sin ciclos) para proporcionar una ruta entre cada par de nodos. Una red como ésta es conocida como un *árbol de expansión*.

El lado izquierdo de la figura 13.7 muestra una red con cinco nodos, en la cual las líneas punteadas representan las ligaduras potenciales que podrían insertarse en la red, mientras que el

■ FIGURA 13.7

a) Datos de un problema de árbol de expansión mínima antes de escoger las ligaduras que deben incluirse en la red y b) solución óptima de este problema donde las líneas oscuras representan las ligaduras seleccionadas.



número a continuación de cada línea punteada representa el costo asociado con la inserción de esa ligadura en particular. Por tanto, el problema es determinar las cuatro ligaduras (sin ciclos) que deben insertarse en la red para minimizar el costo total de dichas ligaduras. El lado derecho de la figura presenta el *árbol de expansión mínima* deseado, donde las líneas oscuras representan las ligaduras que se han insertado en la red con un costo total de 50. Esta solución óptima se obtiene con facilidad al aplicar el algoritmo “codicioso” presentado en la sección 9.4.

Para ilustrar el uso de la búsqueda tabú, a continuación se agregará un par de complicaciones a este ejemplo al suponer que también deben observarse las siguientes restricciones en el momento de elegir las ligaduras a incluirse en la red.

Restricción 1: La ligadura AD se puede incluir sólo si también se incluye la ligadura DE.

Restricción 2: Se puede incluir, como máximo, una de las tres ligaduras: AD, CD y AB.

Observe que la solución óptima previa en el lado derecho de la figura 13.7 viola ambas restricciones porque: 1) está incluida la ligadura AD aunque DE no lo está y 2) se incluye tanto AD como AB.

Al imponer estas restricciones, el algoritmo codicioso que se presentó en la sección 9.4 ya no se puede utilizar para encontrar la nueva solución óptima. Para un problema tan pequeño, es probable que esta solución se encuentre con mayor rapidez mediante inspección. Sin embargo, a continuación se verá cómo podría utilizarse la búsqueda tabú, tanto en éste como en problemas mucho más grandes, para buscar una solución óptima.

La manera más fácil de tomar en cuenta estas restricciones es cargar una penalización muy grande por violarlas, como las que se presentan en seguida.

1. Cargar una penalización de 100 si se viola la restricción 1.
2. Cargar una penalización de 100 si se incluyen dos de las tres ligaduras especificadas en la restricción 2. Aumentar esta penalización a 200 si se incluyen las tres ligaduras.

Una penalización de 100 es lo suficientemente grande como para asegurar que las restricciones no se violarán en un árbol de expansión mínima que minimice el costo total, incluyendo las penalizaciones, dado que existen algunas restricciones factibles. Al duplicar la penalización en caso de que la restricción 2 se viole en mayor medida, se proporciona un incentivo para al menos reducir el número de ligaduras que se consideran durante una iteración de la búsqueda tabú.

Existen algunas formas de responder las seis preguntas que se necesitan para especificar la manera en que se realizará una búsqueda tabú. (Vea la lista de preguntas que sigue al esquema de un algoritmo de búsqueda tabú básico.) A continuación se presenta una forma directa de contestar las preguntas.

1. **Procedimiento de búsqueda local:** En cada iteración elija el mejor vecino inmediato de la solución de prueba actual que no esté descartado por la lista tabú.
2. **Estructura de vecindad:** Un vecino inmediato de la solución de prueba actual es aquél al que se llega mediante la adición de una sola ligadura y la eliminación de una de las otras ligaduras dentro del ciclo que se forma por la adición realizada. (La ligadura que se borra debe formar parte de este ciclo para así conservar la forma de un árbol de expansión.)
3. **Forma de los movimientos tabú:** Enumere las ligaduras que no se deben eliminar.
4. **Adición de un movimiento tabú:** En cada iteración, después de elegir la ligadura que debe agregarse a la red, también incorpore esta ligadura a la lista tabú.
5. **Tamaño máximo de la lista tabú:** Dos. Siempre que se agregue un movimiento tabú a una lista llena, elimine el más antiguo de los dos movimientos tabú que ya estaban en la lista. (Como un árbol de expansión del problema bajo consideración sólo incluye cuatro ligaduras, la lista tabú debe ser muy pequeña para proporcionar algo de flexibilidad al elegir la ligadura que debe borrarse en cada iteración.)
6. **Regla de detención:** Detenga el proceso después de tres iteraciones consecutivas sin mejora del mejor valor de la función objetivo. (También debe detenerse en cualquier iteración donde la solución de prueba actual no tenga vecinos inmediatos que no estén descartados por la lista tabú.)

Después de haber especificado estos detalles, se puede proceder a aplicar el algoritmo de búsqueda tabú al ejemplo. Para comenzar, una elección razonable de la solución de prueba inicial

es la solución óptima de la versión no restringida del problema que se muestra en la figura 13.7b). Como esta solución viola ambas restricciones (pero con la inclusión de sólo dos de las tres ligaduras especificadas en la restricción 2), es necesario imponer penalizaciones de 100 dos veces. Por lo tanto, el costo total de esta solución es

$$\begin{aligned}\text{Costo} &= 20 + 10 + 5 + 15 + 200 \text{ (penalizaciones de restricción)} \\ &= 250.\end{aligned}$$

Iteración 1. Las tres opciones para agregar una ligadura a la red de la figura 13.7b) son BE, CD y DE. Si se eligiera BE, el ciclo formado sería BE-CE-AC-AB, en cuyo caso las tres opciones para borrar una ligadura serían CE, AC y AB. (En este punto todavía no se agrega ninguna ligadura a la lista tabú.) Si se borrara CE, el cambio en el costo sería $30 - 5 = 25$ sin cambio en las penalizaciones de restricción, por lo que el costo se incrementaría de 250 a 275. De forma similar, si se borrara AC, el costo total aumentaría de 250 a $250 + (30 - 10) = 270$. Sin embargo, si la ligadura borrada fuese AB, los costos de ligadura cambiarían en $30 - 20 = 10$ y las penalizaciones de restricción disminuirían de 200 a 100 porque ya no se violaría la restricción 2, por lo cual el costo total ahora sería $50 + 10 + 100 = 160$. Estos resultados se resumen en los primeros tres renglones de la tabla 13.1.

En los siguientes dos renglones se resumen los cálculos si se agregara la ligadura CD a la red. En este caso el ciclo creado es CD-AD-AC, de manera que AD y AC son las únicas ligaduras que pueden eliminarse. AC sería una elección particularmente mala porque se seguiría violando la restricción 1 (con penalización de 100), y además se debería cargar una penalización de 200 debido a que se violaría la restricción 2 puesto que las tres ligaduras especificadas en dicha restricción estarían incluidas en la red. Al borrar AD se obtendría el beneficio de satisfacer la restricción 1 y no se incrementaría el grado en el que se viola la restricción 2.

Los últimos tres renglones de la tabla muestran las opciones si se agregara la ligadura DE. El ciclo creado por esta adición sería DE-CE-AC-AD, por lo que CE, AC y AD serían las opciones de eliminación. Las tres satisfarían la restricción 1, pero al borrar AD se cumpliría también con la restricción 2. Si se eliminan por completo las penalizaciones de restricción, el costo total de esta opción sería sólo de $50 + (40 - 15) = 75$. Como éste es el costo más pequeño de las ocho opciones disponibles de movimiento hacia un vecino inmediato a partir de la solución de prueba actual, se selecciona esta acción particular al agregar DE y eliminar AD. Esta elección se indica en la parte dedicada a la iteración 1 de la figura 13.8, mientras que el árbol de expansión resultante, con el cual se inicia la iteración 2, se muestra a la derecha.

Para completar la iteración, como DE se agregó a la red, se convierte en la primera ligadura colocada en la lista tabú. Esta inclusión evitará que DE sea eliminada en la siguiente iteración y que se regrese a la solución de prueba que inició este proceso.

Para resumir, durante la primera iteración se tomaron las siguientes decisiones.

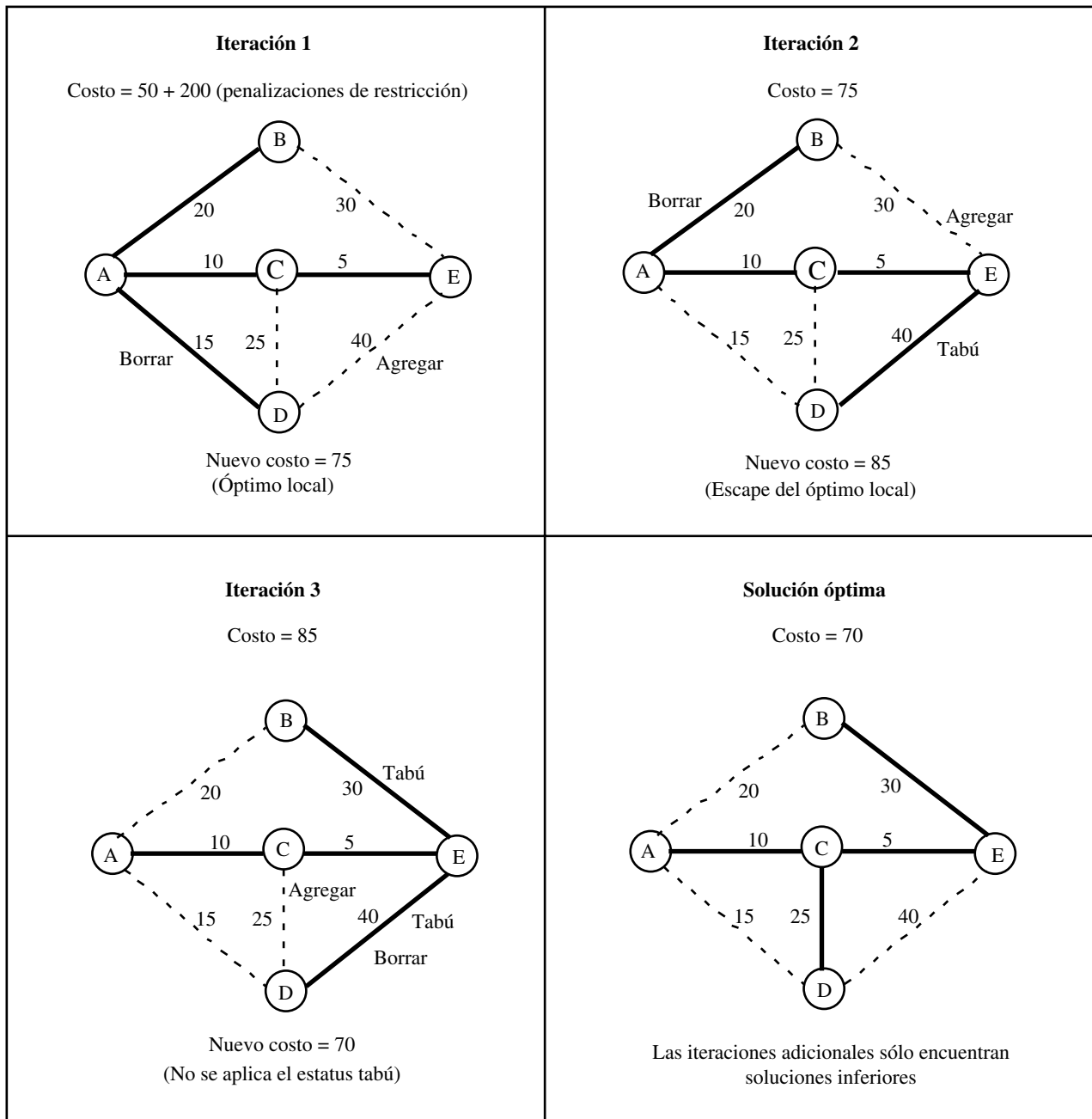
Se agrega la ligadura DE a la red.

Se elimina la ligadura AD de la red.

Se añade la ligadura DE a la lista tabú.

■ **TABLA 13.1** Opciones para agregar y borrar ligaduras en la iteración 1

Agregar	Borrar	Costo
BE	CE	$75 + 200 = 275$
BE	AC	$70 + 200 = 270$
BE	AB	$60 + 100 = 160$
CD	AD	$60 + 100 = 160$
CD	AC	$65 + 300 = 365$
DE	CE	$85 + 100 = 185$
DE	AC	$80 + 100 = 180$
DE	AD	$75 + 0 = 75 \leftarrow \text{Mínimo}$



■ **FIGURA 13.8**

Aplicación de un algoritmo de búsqueda tabú al problema de árbol de expansión mínima que se presentó en la figura 13.7, después de agregarle dos restricciones.

Iteración 2. La parte superior derecha de la figura 13.8 indica que las decisiones correspondientes que se tomaron durante la iteración 2 son las siguientes:

Se agrega la ligadura BE a la red.

De manera automática, esta ligadura agregada se coloca en la lista tabú.

Se elimina la ligadura AB de la red.

En la tabla 13.2 se resumen los cálculos que condujeron a estas decisiones al encontrar que el movimiento del sexto renglón proporciona el menor costo.

■ **TABLA 13.2** Opciones para agregar y borrar ligaduras en la iteración 2

Agregar	Borrar	Costo
AD	DE*	(Movimiento tabú)
AD	CE	$85 + 100 = 185$
AD	AC	$80 + 100 = 180$
BE	CE	$100 + 0 = 100$
BE	AC	$95 + 0 = 95$
BE	AB	$85 + 0 = 85 \leftarrow \text{Mínimo}$
CD	DE*	$60 + 100 = 160$
CD	CE	$95 + 100 = 195$

*Movimiento tabú. Será tomado en cuenta sólo si genera una mejor solución que la mejor solución encontrada con anterioridad.

Los movimientos que se mencionan en los renglones primero y séptimo de la tabla implican la eliminación de DE, lo que está en la lista tabú. Por tanto, estos movimientos se hubieran considerado sólo si hubiesen proporcionado una mejor solución que la mejor solución de prueba encontrada hasta el momento, la cual tiene un costo de 75 dólares. El cálculo en el séptimo renglón muestra que este movimiento no proporcionaría una mejor solución. En el caso del primer renglón ni siquiera es necesario realizar un cálculo porque este movimiento regresaría el proceso a la solución de prueba anterior.

Observe que el movimiento del sexto renglón se realiza aunque genera una nueva solución de prueba con un costo más elevado (85) que el de la solución anterior (75) con la que se inició la iteración 2. Esto significa que la solución de prueba anterior fue un óptimo local porque todos sus vecinos inmediatos (aquellos a los que se puede llegar mediante uno de los movimientos que se mencionan en la tabla 13.2) tienen un costo más alto. Sin embargo, al desplazarse hacia el mejor de los vecinos inmediatos, el proceso tiene la posibilidad de escapar del óptimo local y continuar la búsqueda del óptimo global.

Antes de continuar con la iteración 3 es necesario hacer una observación sobre las formas más avanzadas de búsqueda tabú que se puede usar para seleccionar el mejor vecino inmediato. Algunos métodos más generales de búsqueda tabú pueden cambiar el significado de “el mejor vecino”, lo cual depende de la historia. Éstos pueden utilizar formas adicionales de memoria para apoyar procesos de intensificación y diversificación. Como ya se mencionó, la intensificación enfoca la búsqueda en una región particularmente promisoría en cuanto a soluciones, mientras que la diversificación conduce la búsqueda a nuevas regiones promisorias.

Iteración 3. En la parte inferior izquierda de la figura 13.8 se resumen las decisiones que se tomaron durante la iteración 3.

Se agrega la ligadura CD a la red.

De manera automática, esta ligadura agregada se coloca en la lista tabú.

Se elimina la ligadura DE de la red.

En la tabla 13.3 se muestra que este movimiento conduce al mejor vecino inmediato de la solución de prueba con la que se inició esta iteración.

Una característica interesante de este movimiento es que se realiza aunque sea un movimiento tabú. La razón por la que se hace es que, además de ser el mejor vecino inmediato, también genera una solución que es mejor (con un costo de 70) que la mejor solución de prueba que se encontró con anterioridad (con un costo de 75). Esto permite que el estatus tabú del movimiento se pase por alto. (La búsqueda tabú también puede incorporar una variedad de criterios más avanzados para ignorar el estatus tabú.)

Antes de comenzar la siguiente iteración es necesario hacer otro ajuste a la lista tabú.

Se borra la ligadura DE de la lista tabú.

■ **TABLA 13.3** Opciones para agregar y borrar ligaduras en la iteración 3

Agregar	Borrar	Costo
AB	BE*	(Movimiento tabú)
AB	CE	$100 + 0 = 100$
AB	AC	$95 + 0 = 95$
AD	DE*	$60 + 100 = 160$
AD	CE	$95 + 0 = 95$
AD	AC	$90 + 0 = 90$
CD	DE*	$70 + 0 = 70 \leftarrow \text{Mínimo}$
CD	CE	$105 + 0 = 105$

*Movimiento tabú. Será tomado en cuenta sólo si genera una mejor solución que la mejor solución de prueba encontrada anteriormente.

Este paso se da por dos razones. Primero, la lista tabú consiste en ligaduras que por lo general no se deben borrar de la red durante la iteración actual (con la excepción que se mencionó), pero DE ya no está en la red. Segundo, como el tamaño de la lista tabú se estableció en dos y hay otras dos ligaduras (BE y CD) que se han agregado a la lista más recientemente, de cualquier manera DE tendría que haberse borrado de manera automática en este punto.

Continuación. La solución de prueba actual que se muestra en la parte inferior derecha de la figura 13.8 es, en realidad, la solución óptima (el óptimo global) del problema. Sin embargo, el algoritmo de búsqueda tabú no tiene forma de saberlo, por lo que continuará con el proceso de búsqueda. La iteración 4 comenzaría con esta solución de prueba y con las ligaduras BE y CD en la lista tabú. Después de completar esta iteración y dos más, el algoritmo terminaría porque después de tres iteraciones consecutivas no habría mejoría sobre el mejor valor previo de la función objetivo (un costo de 70).

Con un algoritmo de búsqueda tabú bien diseñado, es probable que la mejor solución de prueba que se encuentre después de haber realizado un número modesto de iteraciones sea una buena solución factible. Podría incluso ser una solución óptima, pero no existe garantía de ello. Seleccionar una regla de detención que proporcione una corrida relativamente de largo plazo del algoritmo incrementa las posibilidades de alcanzar el óptimo global.

Después de haber explicado la aplicación de este algoritmo mediante su ejecución en un ejemplo pequeño, ahora se aplicará un algoritmo de búsqueda tabú similar al ejemplo del agente viajero que se presentó en la sección 13.1.

Ejemplo del problema del agente viajero

Existen algunos paralelismos cercanos entre un problema de árbol de expansión mínima y el del agente viajero. En ambos casos, el problema consiste en elegir las ligaduras que deben incluirse en la solución. (Recuerde que una solución de un problema del agente viajero puede describirse como la secuencia de ligaduras por las que pasa el agente a lo largo de su recorrido por las ciudades.) En ambos casos, el objetivo es minimizar el costo o la distancia total asociada con el número fijo de ligaduras que se incluye en la solución. En ambos casos existe un procedimiento de búsqueda local intuitivo que implica la adición y eliminación de ligaduras en la solución de prueba actual para obtener la nueva solución de prueba.

En el caso de los problemas de árbol de expansión mínima, el procedimiento de búsqueda local que se describió en la subsección anterior implica la adición y la eliminación de *una sola* ligadura en cada iteración. El procedimiento correspondiente que se explicó en la sección 13.1 del problema del agente viajero implica la utilización de *subviajes inversos* para agregar y borrar un *par* de ligaduras en cada iteración.

Debido a los paralelismos cercanos entre estos dos tipos de problemas, el diseño de un algoritmo de búsqueda tabú para problemas del agente viajero puede ser muy similar al que se acaba de describir para el ejemplo del problema de árbol de expansión mínima. En particular, si se usa el

esquema del algoritmo de búsqueda tabú básico que se presentó antes, es posible responder de una manera similar las seis preguntas que siguen al esquema, como se indica a continuación.

1. **Algoritmo de búsqueda local:** En cada iteración seleccione el mejor vecino inmediato de la solución de prueba actual que no esté descartado por la lista tabú.
2. **Estructura de vecindad:** Un vecino inmediato de la solución de prueba actual es aquel al que se llega por medio de un *subviaje inverso*, como se describe en la sección 13.1 y se ilustra en la figura 13.5. Tal inversión requiere la adición de dos ligaduras y la eliminación de otras dos ligaduras de la solución de prueba actual. (Se descarta un subviaje inverso que sólo invierta la dirección del viaje que proporciona la solución de prueba actual.)
3. **Forma de los movimientos tabú:** Enumere las ligaduras de forma que un subviaje inverso sea tabú si *las dos* ligaduras que se eliminan por esta inversión se encuentran en la lista. (Ello evitará que se regrese con rapidez a la solución de prueba anterior.)
4. **Adición de un movimiento tabú:** En cada iteración, después de elegir las dos ligaduras que deben agregarse a la solución de prueba actual, también incorpore estas dos ligaduras a la lista tabú.
5. **Tamaño máximo de la lista tabú:** Cuatro (dos de cada una de las dos iteraciones más recientes). Siempre que se agregue un par de ligaduras a una lista llena, elimine las dos ligaduras que han permanecido por más tiempo en la lista.
6. **Regla de detención:** Detenga el proceso después de tres iteraciones consecutivas sin mejora del mejor valor de la función objetivo. (También debe detenerse en cualquier iteración donde la solución de prueba actual no tenga vecinos inmediatos que no estén descartados por la lista tabú.)

Para aplicar este algoritmo de búsqueda tabú al ejemplo (vea la figura 13.4), se comenzará con la misma solución de prueba inicial, 1-2-3-4-5-6-7-1, como en la sección 13.1. Recuerde cómo al empezar el algoritmo del subviaje inverso (un algoritmo de mejora local), con esta solución de prueba inicial se llega en dos iteraciones (vea las figuras 13.5 y 13.6) a un óptimo local en 1-2-4-6-5-3-7-1, en cuyo punto el algoritmo se detiene. Con la excepción de que se crea una lista tabú, el algoritmo de búsqueda tabú se inicia exactamente del mismo modo, como se resume a continuación.

Solución de prueba inicial: 1-2-3-4-5-6-7-1 Distancia = 69
 Lista tabú: En blanco.

Iteración 1: Se elige invertir 3-4 (vea la figura 13.5).
 Ligaduras borradas: 2-3 y 4-5
 Ligaduras agregadas: 2-4 y 3-5
 Lista tabú: Ligaduras 2-4 y 3-5.
 Nueva solución de prueba: 1-2-4-3-5-6-7-1 Distancia = 65

Iteración 2: Se elige invertir 3-5-6 (vea la figura 13.6).
 Ligaduras borradas: 4-3 y 6-7 (no están en la lista tabú)
 Ligaduras agregadas: 4-6 y 3-7
 Lista tabú: Ligaduras 2-4, 3-5, 4-6 y 3-7
 Nueva solución de prueba: 1-2-4-6-5-3-7-1 Distancia = 64

Sin embargo, en lugar de terminar, el algoritmo de búsqueda tabú escapa de este óptimo local (que se muestra en el lado derecho de la figura 13.6 y en el lado izquierdo de la figura 13.9) pues se traslada hacia el mejor vecino inmediato de la solución de prueba actual aunque su distancia sea mayor. Debido a la disponibilidad limitada de ligaduras entre pares de nodos (ciudades) de la figura 13.4, la solución de prueba actual sólo tiene dos vecinos inmediatos, los cuales se describen a continuación.

Si se invierte 6-5-3: 1-2-4-3-5-6-7-1 Distancia = 65
 Si se invierte 3-7: 1-2-4-6-5-7-3-1 Distancia = 66

(Se descarta invertir 2-4-6-5-3-7 para obtener 1-7-3-5-6-4-2-1 porque es exactamente el mismo viaje pero en la dirección opuesta.) Sin embargo, se debe descartar el primero de estos vecinos inmediatos porque requeriría eliminar las ligaduras 4-6 y 3-7, lo cual está prohibido porque *las dos* ligaduras se encuentran en la lista tabú. (Este movimiento se podría permitir si implicara alguna mejoría en la mejor solución de prueba que se encontró hasta ahora, pero no es así.) Al descartar este vecino inmediato simplemente se evita regresar a la solución de prueba anterior. Entonces, por no existir otro, se selecciona el segundo de los vecinos inmediatos como la próxima solución de prueba, como se resume a continuación.

Iteración 3: Se elige invertir 3-7 (vea la figura 13.9).

Ligaduras borradas: 5-3 y 7-1

Ligaduras agregadas: 5-7 y 3-1

Lista tabú: 4-6, 3-7, 5-7 y 3-1

(2-4 y 3-5 se eliminan de la lista)

Nueva solución de prueba: 1-2-4-6-5-7-3-1 Distancia = 66

La inversión del subviaje de esta iteración se puede ver en la figura 13.9 en la cual las líneas punteadas muestran las ligaduras borradas (a la izquierda) y las agregadas (a la derecha) para obtener la nueva solución de prueba. Observe que una de las ligaduras borradas es 5-3 aun cuando estaba en la lista tabú al final de la iteración 2. Esto es correcto porque un subviaje inverso es tabú sólo si *las dos* ligaduras borradas están en la lista tabú. También observe que la lista tabú actualizada al final de la iteración 3 ya no contiene las dos ligaduras que habían permanecido por más tiempo en dicha lista (las que se agregaron durante la iteración 1), puesto que el tamaño máximo de la lista tabú se estableció en cuatro.

La nueva solución de prueba tiene los cuatro vecinos inmediatos que se describen a continuación.

Invertir 2-4-6-5-7: 1-7-5-6-4-2-3-1 Distancia = 65

Invertir 6-5: 1-2-4-5-6-7-3-1 Distancia = 69

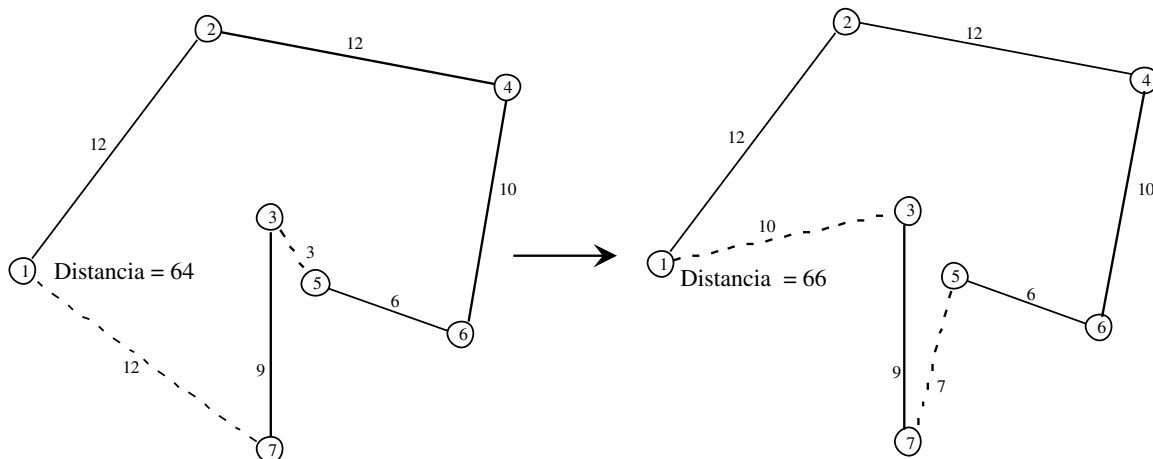
Invertir 5-7: 1-2-4-6-5-7-3-1 Distancia = 63

Invertir 7-3: 1-2-4-6-5-3-7-1 Distancia = 64

Sin embargo, el segundo de estos vecinos inmediatos es tabú porque *las dos* ligaduras borradas (4-6 y 5-7) están en la lista tabú. Por la misma razón, el cuarto vecino inmediato (que es la solución de prueba anterior) también es tabú. Por lo tanto, las únicas opciones viables son el primero y el tercero de los vecinos inmediatos. Como el último de éstos tiene la distancia más corta, se convierte en la próxima solución de prueba, como se resume a continuación.

■ FIGURA 13.9

Inversión del subviaje 3-7 en la iteración 3 que conduce desde la solución de prueba de la izquierda hasta la nueva solución de prueba de la derecha.



Iteración 4: Se elige invertir 5-7 (vea la figura 13.10).

Ligaduras borradas: 6-5 y 7-3

Ligaduras agregadas: 6-7 y 5-3

Lista tabú: 5-7, 3-1, 6-7 y 5-3

(4-6 y 3-7 se eliminan de la lista)

Nueva solución de prueba: 1-2-4-6-7-5-3-1

Distancia = 63

En la figura 13.10 se muestra esta inversión del subviaje. El viaje de la nueva solución de prueba a la derecha tiene una distancia de sólo 63, que es menor que cualquiera de las soluciones anteriores. En realidad, esta nueva solución resulta ser la solución óptima.

Si no se sabe esto, el algoritmo de búsqueda tabú intentará ejecutar más iteraciones. Sin embargo, el único vecino inmediato de la solución de prueba actual es la solución de prueba que se obtuvo en la iteración anterior. Esto requeriría eliminar las ligaduras 6-7 y 5-3, que se encuentran en la lista tabú, de manera que es imposible regresar a la solución de prueba anterior. Como ya no existe otro vecino inmediato disponible, la regla de detención da por finalizado el algoritmo en este punto con 1-2-4-6-7-5-3-1 (la mejor de las soluciones de prueba) como la solución final. Aunque no hay garantía de que la solución final del algoritmo sea una solución óptima, resulta afortunado que en este caso la solución haya resultado un óptimo global.

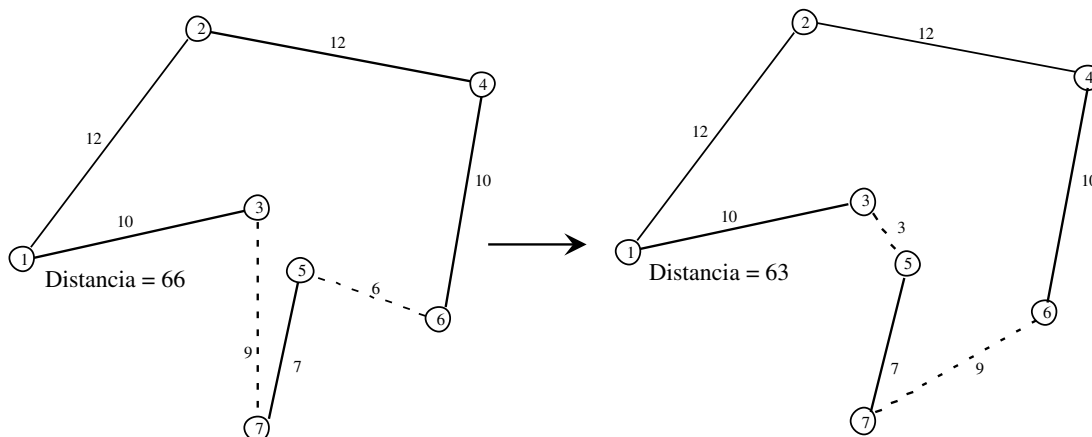
En el área de metaheurísticas del IOR Tutorial se incluye una rutina para aplicar este algoritmo de búsqueda tabú particular a otros pequeños problemas del agente viajero.

Este algoritmo en específico es sólo un ejemplo de un algoritmo de búsqueda tabú posible para manejar problemas del agente viajero. Varios detalles del algoritmo pueden modificarse en diversas formas razonables. Por ejemplo, por lo general el método no se detiene cuando todos los movimientos disponibles están prohibidos por la lista tabú, sino que selecciona el movimiento “menos tabú”. También, una característica importante de los métodos de búsqueda tabú generales es el uso de vecindades múltiples, que descansa en vecindades básicas cuando comienza el proceso para después incluir vecindades más avanzadas cuando la tasa a la que se encuentran soluciones mejoradas disminuye. El elemento adicional más significativo de la búsqueda tabú es el uso de estrategias de intensificación y diversificación, que se mencionó con anterioridad. Sin embargo, el esquema básico general de un enfoque de búsqueda tabú con “memoria a corto plazo”, en lo general, se conserva igual que como se ha ilustrado aquí.

Los dos ejemplos considerados en esta sección caen en la categoría de problemas de optimización combinatoria que involucran redes. Ésta es un área de aplicación particularmente común de los algoritmos de búsqueda tabú. El esquema general de estos algoritmos incorpora los principios que se presentaron en esta sección, pero los detalles se trabajan para ajustarse a la estructura del problema específico bajo consideración.

■ FIGURA 13.10

Inversión del subviaje 5-7 en la iteración 4 que conduce desde la solución de prueba de la izquierda hasta la nueva solución de prueba de la derecha (que resulta ser la solución óptima).



13.3 TEMPLADO SIMULADO

El templado simulado es otra metaheurística que se utiliza con amplitud que permite al proceso de búsqueda escapar de un óptimo local. Para compararlo y contrastarlo mejor con la búsqueda tabú, se aplicará al mismo problema del agente viajero antes de regresar al ejemplo de programación no lineal que se explicó en la sección 13.1. Pero primero se examinarán los conceptos básicos del templado simulado.

Conceptos básicos

En la figura 13.1 de la misma sección se introdujo el concepto de que la localización del óptimo global de un problema complicado de maximización es análogo a determinar cuál de todas las montañas existentes es la más alta y después escalar hasta la cumbre de esta montaña. Desafortunadamente, un proceso de búsqueda matemática no tiene el beneficio de una visión aguda que pudiera ayudar a rastrear una cumbre a la distancia. En su lugar, el proceso consiste en algo parecido a escalar en medio de una niebla densa donde la única pista de la dirección que debe tomarse es qué tan pronunciada es la pendiente ascendente o descendente del siguiente paso.

Un enfoque, adoptado por la búsqueda tabú, es escalar la pendiente actual en la dirección más empinada hasta alcanzar ese pico particular y después descender lentamente para buscar otra cuesta que escalar. La desventaja es que se emplea demasiado tiempo (iteraciones) en escalar cada cuesta que se encuentra en lugar de buscar la más alta.

Por otro lado, el enfoque que se utiliza en el templado simulado es concentrarse de manera principal en la búsqueda de la cuesta más alta. Como el pico más alto puede estar en cualquier sitio dentro de la región factible, se le otorga la mayor importancia a dar pasos en direcciones aleatorias (excepto por el rechazo de algunos, pero no de todos, los pasos que descienden en lugar de ascender) con la intención de explorar tanta región factible como sea posible. Como la mayoría de los pasos aceptados son ascendentes, en forma gradual la búsqueda gravitará hacia aquellas partes de la región factible que contienen las cumbres más altas. Por lo tanto, de manera gradual, el proceso de búsqueda incrementa el hincapié en el ascenso al rechazar una proporción creciente de pasos que descienden. Si se da el tiempo suficiente, con frecuencia el proceso encontrará y ascenderá hasta la cuesta más alta.

De manera más específica, cada iteración del proceso de búsqueda de templado simulado se mueve de la solución de prueba actual a un vecino inmediato en la vecindad local de esta solución, igual que en la búsqueda tabú. Sin embargo, la diferencia con ésta reside en cómo se selecciona un vecino inmediato para ser la próxima solución de prueba. Sea

Z_c = valor de la función objetivo de la solución de prueba *actual*,

Z_n = valor de la función objetivo del candidato actual a ser la siguiente solución de prueba,

T = un parámetro que mide la tendencia a aceptar el candidato actual para ser la próxima solución de prueba si este candidato no es una mejora sobre la solución de prueba actual.

La regla para seleccionar cuál vecino inmediato será la próxima solución de prueba es la siguiente.

Regla de selección del movimiento: Entre todos los vecinos inmediatos de la solución de prueba actual, seleccione uno de manera aleatoria para convertirse en el candidato actual a ser la próxima solución de prueba. Si se supone que el objetivo es la *maximización* de la función objetivo, acepte o rechace este candidato para ser la próxima solución de prueba como sigue:

Si $Z_n \geq Z_c$, siempre acepte este candidato.

Si $Z_n < Z_c$, acepte el candidato con la siguiente probabilidad:

$$\text{Prob}\{\text{aceptación}\} = e^x \text{ donde } x = \frac{Z_n - Z_c}{T}$$

(Si el objetivo es de *minimización*, se deben invertir Z_n y Z_c en las fórmulas anteriores.)

Si el candidato es rechazado repita el proceso con un vecino inmediato de la solución de prueba actual seleccionado de manera aleatoria. (Si ya no existen vecinos inmediatos restantes, el algoritmo termina.)

■ **TABLA 13.4** Algunas probabilidades de muestra de que la regla de selección del movimiento acepte un paso descendente cuando el objetivo es maximizar

$x = \frac{Z_n - Z_c}{T}$	Prob{aceptación} = e^x
-0.01	0.990
-0.1	0.905
-0.25	0.779
-0.5	0.607
-1	0.368
-2	0.135
-3	0.050
-4	0.018
-5	0.007

Por tanto, si el candidato actual bajo consideración es mejor que la solución de prueba actual, éste siempre se acepta como la próxima solución de prueba. Si es peor, la probabilidad de aceptación depende de qué tan peor es (y del tamaño de T). En la tabla 13.4 se presenta una muestra de estos valores de probabilidad, que van desde una muy alta cuando el candidato actual es sólo un poco peor (en relación con T) que la solución de prueba actual hasta una probabilidad en extremo pequeña cuando es mucho peor. En otras palabras, la regla de selección del movimiento por lo general aceptará un paso que sólo es un poco descendente, pero será muy difícil que acepte un paso descendente muy pronunciado. Si se inicia con un valor relativamente grande de T (como lo hace el templado simulado) la probabilidad de aceptación correspondiente será alta, lo que permite a la búsqueda proceder en direcciones casi aleatorias. Si se disminuye de manera gradual el valor de T a medida que la búsqueda continúa (como lo hace el templado simulado) la probabilidad de aceptación disminuye de un modo paulatino, lo cual aumenta la importancia de ascender la mayor parte del tiempo. Así, la elección de los valores de T a través del tiempo controla el grado de aleatoriedad del proceso para permitir pasos descendentes. Este componente aleatorio, que no está presente en la búsqueda tabú básica, proporciona más flexibilidad de movimiento hacia otra parte de la región factible con la esperanza de encontrar una cumbre más alta.

El método usual de implantar la regla de selección del movimiento para determinar si se aceptará un paso descendente particular es comparar un **número aleatorio** entre 0 y 1 con la probabilidad de aceptación. Ese número aleatorio puede considerarse como una observación aleatoria en una distribución uniforme entre 0 y 1. (A lo largo de este capítulo todas las referencias a números aleatorios tendrán relación con ese tipo de números aleatorios.) Existen varios métodos para generar estos números aleatorios (como se describirá en la sección 20.3). Por ejemplo, la función de Excel ALEATORIO() los genera cuando se le solicita. (También, al inicio de la sección de problemas se describe cómo pueden usarse los dígitos aleatorios de la tabla 20.3 para obtener los números aleatorios que se necesitarán para resolver algunos de los problemas de tarea.)

Si *número aleatorio* < Prob{aceptación}, se acepta un paso descendente.
En otro caso, se rechaza el paso.

¿Por qué el templado simulado usa la fórmula particular para Prob{aceptación} que especifica la regla de selección del movimiento? La razón es que el templado simulado se basa en la analogía con un *proceso de templado físico*. En principio, este proceso implica fundir un metal o cristal a una temperatura alta para después enfriar lentamente la sustancia hasta que alcance un estado estable de energía con propiedades físicas deseables. A una temperatura dada T durante este proceso, el nivel de energía de los átomos de la sustancia fluctúa pero tiende a disminuir. Un modelo matemático de cómo fluctúa el nivel de energía supone que los cambios ocurren de manera aleatoria excepto que sólo se aceptan algunos de los incrementos. En particular, la probabilidad de aceptar un incremento cuando la temperatura es T tiene la misma forma que para Prob{aceptación} en la regla de selección del movimiento del templado simulado.

La analogía con un problema de optimización en la forma de minimización es que el nivel de energía de la sustancia en el estado actual del sistema corresponde al valor de la función objetivo

con la solución factible actual del problema. El objetivo de que la sustancia alcance un nivel estable con un nivel de energía tan pequeño como sea posible corresponde al problema de llegar a una solución factible con un valor de la función objetivo tan pequeño como sea posible.

Igual que en un proceso físico de templado, una cuestión clave cuando se diseña un algoritmo de templado simulado de un problema de optimización es seleccionar un **programa de temperatura** apropiado para ser usado. (Debido a la analogía con el templado físico, en el algoritmo de templado simulado se hará referencia a T como la temperatura.) Este programa debe especificar el valor inicial de T , relativamente grande, así como los valores subsecuentes que se reducen en forma progresiva. También debe precisar cuántos movimientos (iteraciones) se deben hacer para cada valor de T . La selección de estos parámetros para ajustarse al problema bajo consideración es un factor clave de la eficacia del algoritmo. Se puede utilizar cierta experimentación preliminar para guiar la selección de los parámetros del algoritmo. Después se especificará un programa de temperatura específico que parece razonable para los dos ejemplos que se consideran en esta sección, pero también se podrían aplicar muchos otros.

Con esta base, ahora es posible proporcionar un esquema del algoritmo de templado simulado básico.

Bosquejo de un algoritmo de templado simulado básico

Paso inicial. Comience con una solución de prueba inicial.

Iteración. Utilice la *regla de selección del movimiento* para elegir la próxima solución de prueba. (Si ninguno de los vecinos inmediatos de la solución de prueba actual se acepta, el algoritmo termina.)

Verificación del programa de temperatura. Después de realizar el número deseado de iteraciones en el valor actual de T , disminuya T hasta el siguiente valor en el programa de temperatura y reinicie las iteraciones en dicho valor.

Regla de detención. Cuando ya se ha realizado el número de iteraciones deseado en el valor más pequeño de T en el programa de temperatura (o cuando no se acepta ninguno de los vecinos inmediatos de la solución de prueba actual), se detiene el algoritmo. Acepte la mejor solución de prueba que encontró en cualquier iteración (incluso de los valores grandes de T) como la solución final.

Antes de aplicar este algoritmo a cualquier problema particular, es necesario trabajar cierto número de detalles para ajustarse a la estructura del problema.

1. ¿Cómo debe seleccionarse la solución de prueba inicial?
2. ¿Cuál es la *estructura de vecindad* que especifica las soluciones que son vecinos inmediatos de cualquier solución de prueba actual (a los que se puede llegar en una sola iteración)?
3. ¿Qué dispositivo debe usarse en la regla de selección del movimiento para elegir en forma *aleatoria* uno de los vecinos inmediatos de la solución de prueba actual para convertirlo en el candidato actual a ser la próxima solución de prueba?
4. ¿Cuál es el programa de temperatura apropiado?

Se ilustrarán algunas formas razonables de contestar estas preguntas en el contexto de la aplicación del algoritmo de templado simulado a los siguientes dos ejemplos.

Ejemplo del problema del agente viajero

De nuevo se considerará el problema particular del agente viajero que se introdujo en la sección 13.1 y se desplegó en la figura 13.4.

En el área de metaheurísticas del IOR Tutorial se incluye una rutina para aplicar el algoritmo de templado simulado básico a pequeños problemas del agente viajero como este ejemplo. Esta rutina responde las cuatro preguntas de la siguiente manera:

1. **Solución de prueba inicial:** Se puede introducir cualquier solución factible (secuencia de ciudades en el viaje), quizá al generar en forma aleatoria la secuencia, pero resulta útil introducir

alguna que parezca una buena solución factible. Por ejemplo, la solución factible 1-2-3-4-5-6-7-1 es una elección razonable.

2. **Estructura de vecindad:** Un vecino inmediato a la solución de prueba actual es alguno al que se pueda llegar mediante un *subviaje inverso*, como se describe en la sección 13.1 y se ilustra en la figura 13.5. (No obstante, se descarta el subviaje inverso que sólo invierte la dirección del viaje que proporciona la solución de prueba actual.)
3. **Selección aleatoria de un vecino inmediato:** La selección de un subviaje que debe invertirse requiere la selección del espacio en la secuencia actual de ciudades donde actualmente se inicia el subviaje y después el espacio donde ahora termina. El espacio de inicio puede ser cualquiera, excepto el primero y el último (reservados para la ciudad de residencia), y el espacio contiguo al último. El espacio de terminación debe estar en cualquier punto después del espacio inicial, excepto en el último. (Un inicio en el segundo espacio y terminación en el espacio contiguo al último también se descarta puesto que éste sólo invertiría la dirección del viaje.) Como se ilustrará más adelante, se utilizan números aleatorios para obtener probabilidades iguales de selección para cualquiera de los espacios de inicio y de terminación elegibles. Si esta selección de los espacios de inicio y terminación resulta ser no factible (porque las ligaduras necesarias para completar el subviaje inverso no están disponibles), el proceso se repite hasta que se encuentra una selección factible.
4. **Programa de temperatura:** Se realizan cinco iteraciones a cada uno de los cinco valores de $T (T_1, T_2, T_3, T_4, T_5)$, donde

$$\begin{aligned} T_1 &= 0.2Z_c \text{ cuando } Z_c \text{ es el valor de la función objetivo de la solución de prueba inicial,} \\ T_2 &= 0.5T_1, \\ T_3 &= 0.5T_2, \\ T_4 &= 0.5T_3, \\ T_5 &= 0.5T_4, \end{aligned}$$

Este programa de temperatura particular sólo es ilustrativo de lo que se podría utilizar. $T_1 = 0.2Z_c$ es una elección razonable porque T_1 debe tender a ser bastante grande en comparación con los valores típicos de $|Z_n - Z_c|$, lo que estimulará una búsqueda casi aleatoria a través de la región factible para encontrar dónde debe enfocarse la búsqueda. Sin embargo, para el momento en que T se reduce hasta T_5 , casi no se aceptan movimientos sin mejora, por lo que el acento estará en mejorar el valor de la función objetivo.

Cuando se enfrenten problemas más grandes, es probable que se realicen más de cinco iteraciones de cada valor de T . Aún más, es probable que los valores de T se reduzcan con mayor lentitud respecto del programa de temperatura descrito anteriormente.

A continuación se explicará cómo se hace la selección aleatoria de un vecino inmediato. Suponga que se tiene la solución de prueba inicial 1-2-3-4-5-6-7-1 en el ejemplo.

Solución de prueba inicial: 1-2-3-4-5-6-7-1 $Z_c = 69$ $T_1 = 0.2Z_c = 13.8$

El subviaje que debe invertirse puede comenzar en cualquier lugar entre el segundo espacio (que en la actualidad designa a la ciudad 2) y el sexto espacio (que en la actualidad designa a la ciudad 6). A cada uno de estos cinco espacios se le puede otorgar una misma probabilidad mediante la asignación de los siguientes valores de un número aleatorio entre 0 y 1 para elegir el espacio que se indica a continuación.

0.0000-0.1999:	El subviaje se inicia en el espacio 2.
0.2000-0.3999:	El subviaje se inicia en el espacio 3.
0.4000-0.5999:	El subviaje se inicia en el espacio 4.
0.6000-0.7999:	El subviaje se inicia en el espacio 5.
0.8000-0.9999:	El subviaje se inicia en el espacio 6.

Suponga que el número aleatorio generado resultó ser 0.2779.

0.2779: Se elige un subviaje que se inicia en el espacio 3.

Al comenzar en el espacio 3, el subviaje que se invertirá necesita terminar en algún lugar entre los espacios 4 y 7. A estos cuatro espacios se les proporcionan probabilidades iguales mediante la siguiente correspondencia con un número aleatorio.

0.0000-0.2499:	El subviaje se inicia en el espacio 4.
0.2500-0.4999:	El subviaje se inicia en el espacio 5.
0.5000-0.7499:	El subviaje se inicia en el espacio 6.
0.7500-0.9999:	El subviaje se inicia en el espacio 7.

Suponga que el número aleatorio generado para este propósito resultó ser 0.0461.

0.0461: Se elige el final del subviaje en el espacio 4.

Como los espacios 3 y 4 en la actualidad designan que las ciudades 3 y 4 deben ser la tercera y cuarta ciudades visitadas en el viaje, el subviaje entre las ciudades 3 y 4 se invertirá.

Se invierte 3-4 (vea la figura 13.5): 1-2-4-3-5-6-7-1 $Z_n = 65$

Este vecino inmediato de la solución actual (inicial) se convierte en el candidato actual a ser la mejor solución de prueba. Como

$$Z_n = 65 < Z_c = 69,$$

este candidato es mejor que la solución de prueba actual (recuerde que el objetivo aquí es *minimizar* la distancia total del viaje), por lo cual este candidato se acepta de manera automática para ser la siguiente solución de prueba.

Esta selección de un subviaje inverso fue afortunada porque condujo a una solución factible. Esto no sucede siempre en los problemas del agente viajero, como en el ejemplo donde ciertos pares de ciudades no están conectados de manera directa por una ligadura. Por ejemplo, si los números aleatorios hubieran ordenado la inversión de 2-3-4-5 para obtener el viaje 1-5-4-3-2-6-7-1, la figura 13.4 muestra que ésta es una solución no factible porque no existe liga entre las ciudades 1 y 5 así como entre las ciudades 2 y 6. Cuando se presenta esta situación, es necesario generar nuevos pares de números aleatorios hasta que se obtenga una solución factible. (También se puede construir un procedimiento más complejo para generar números aleatorios sólo para las ligaduras relevantes.)

Para ilustrar un caso donde el candidato actual a ser la próxima solución de prueba es peor que la solución de prueba actual, suponga que la segunda iteración resultó en la inversión de 3-5-6 (como en la figura 13.6) para obtener 1-2-4-6-5-3-7-1, que tiene una distancia total de 64. Después suponga que la tercera iteración comienza al invertir 3-7 (como en la figura 13.9) para obtener 1-2-4-6-5-7-3-1 (que tiene una distancia total de 66) como el candidato actual a ser la próxima solución de prueba. Como 1-2-4-6-5-3-7-1 (con una distancia total de 64) es la solución de prueba actual de la iteración 3, ahora se tiene

$$Z_c = 64, \quad Z_n = 66, \quad T_1 = 13.8.$$

Por lo tanto, como el objetivo aquí es *minimizar*, la probabilidad de aceptación de 1-2-4-6-5-7-3-1 como la próxima solución de prueba es

$$\begin{aligned} \text{Prob}\{\text{aceptación}\} &= e^{(Z_c - Z_n)/T_1} \\ &= e^{-2/13.8} \\ &= 0.865. \end{aligned}$$

Si el siguiente número aleatorio que se genera es menor que 0.865, este candidato a solución se debe aceptar como la siguiente solución de prueba. En caso contrario, se debe rechazar.

En la tabla 13.5 se muestran los resultados de la utilización del IOR Tutorial para aplicar por completo el algoritmo de templado simulado a este problema. Observe que las iteraciones 14 y 16 empatan al encontrar la mejor solución de prueba, 1-3-5-7-6-4-2-1 (que resulta ser la solución óptima junto con el viaje equivalente en la dirección inversa, 1-2-4-6-7-5-3-1), de manera que

■ **TABLA 13.5** Aplicación del algoritmo de templado simulado del IOR
Tutorial al ejemplo del problema del agente viajero

Distancia	<i>T</i>	Solución de prueba obtenida	Distancia
0		1-2-3-4-5-6-7-1	69
1	13.8	1-3-2-4-5-6-7-1	68
2	13.8	1-2-3-4-5-6-7-1	69
3	13.8	1-3-2-4-5-6-7-1	68
4	13.8	1-3-2-4-6-5-7-1	65
5	13.8	1-2-3-4-6-5-7-1	66
6	6.9	1-2-3-4-5-6-7-1	69
7	6.9	1-3-2-4-5-6-7-1	68
8	6.9	1-2-3-4-5-6-7-1	69
9	6.9	1-2-3-5-4-6-7-1	65
10	6.9	1-2-3-4-5-6-7-1	69
11	3.45	1-2-3-4-6-5-7-1	66
12	3.45	1-3-2-4-6-5-7-1	65
13	3.45	1-3-7-5-6-4-2-1	66
14	3.45	1-3-5-7-6-4-2-1	63 ← Mínimo
15	3.45	1-3-7-5-6-4-2-1	66
16	1.725	1-3-5-7-6-4-2-1	63 ← Mínimo
17	1.725	1-3-7-5-6-4-2-1	66
18	1.725	1-3-2-4-6-5-7-1	65
19	1.725	1-2-3-4-6-5-7-1	66
20	1.725	1-3-2-4-6-5-7-1	65
21	0.8625	1-3-7-5-6-4-2-1	66
22	0.8625	1-3-2-4-6-5-7-1	65
23	0.8625	1-2-3-4-6-5-7-1	66
24	0.8625	1-3-2-4-6-5-7-1	65
25	0.8625	1-3-7-5-6-4-2-1	66

esta solución se acepta como la solución final. El lector interesado podría encontrar muy útil la aplicación de este software al mismo problema una vez más. Dada la aleatoriedad construida en el algoritmo, la secuencia de soluciones de prueba que se obtenga será diferente cada vez. Debido a esta característica, los practicantes reaplican varias veces un algoritmo de templado simulado al mismo problema para incrementar la oportunidad de encontrar una solución óptima. (En el problema 13.3-2 se pide hacerlo para este mismo ejemplo.) La solución de prueba inicial también puede cambiarse cada vez para facilitar una exploración más profunda de toda la región factible.

Si desea ver **otro ejemplo** de cómo se utilizan los números aleatorios para realizar una iteración del algoritmo de templado simulado básico de un problema del agente viajero, en la sección Worked Examples del sitio en internet de este libro se proporciona uno de ellos.

Antes de continuar con el ejemplo siguiente, se debe hacer una pausa en este punto para mencionar un par de formas en las que las avanzadas características de la búsqueda tabú se pueden combinar de manera fructífera con el templado simulado. Una forma es aplicar la característica de *oscilación estratégica* de la búsqueda tabú al programa de temperatura del templado simulado. Dicha oscilación ajusta el programa de temperatura al mover las temperaturas en un sentido y otro a través de los niveles donde se encontraron las mejores soluciones. Otra forma implica la aplicación de las estrategias de la lista de candidatos de la búsqueda tabú a la regla de selección del movimiento del templado simulado. La idea aquí es identificar múltiples vecinos para ver si se encuentra un movimiento con mejora antes de aplicar la regla aleatorizada para aceptar o rechazar el candidato actual a ser la próxima solución de prueba. Algunas veces, estos cambios producen mejoras significativas.

Como estas ideas para la aplicación de características de la búsqueda tabú al templado simulado lo sugieren, algunas veces un *algoritmo híbrido* que combina las ideas de diferentes metaheurísticas puede desempeñarse mejor que un algoritmo que se basa sólo en una sola metaheurística. Aunque en este capítulo se han presentado las tres metaheurísticas que se usan con mayor frecuencia por separado, en ocasiones los practicantes expertos escogen entre las ideas de éstas y otras metaheurísticas para diseñar sus métodos heurísticos.

Ejemplo de programación no lineal

Ahora reconsidere el ejemplo del problema de programación no lineal pequeño (con una sola variable) que se introdujo en la sección 13.1. El problema consiste en

$$\text{Maximizar } f(x) = 12x^5 - 975x^4 + 28\,000x^3 - 345\,000x^2 + 1\,800\,000x,$$

sujeto a

$$0 \leq x \leq 31.$$

La gráfica de $f(x)$ en la figura 13.1 revela que existen óptimos locales en $x = 5$, $x = 20$ y $x = 31$, pero sólo $x = 20$ es un óptimo global.

El área de metaheurísticas del IOR Tutorial incluye una rutina para aplicar el algoritmo de templado simulado a problemas de programación no lineal pequeños de la forma,

$$\text{Maximizar } f(x_1, \dots, x_n)$$

sujeto a

$$L_j \leq x_j \leq U_j, \text{ para } j = 1, \dots, n,$$

donde $n = 1$ o 2 , y donde L_j y U_j son constantes ($0 \leq L_j < U_j \leq 63$) que representan los límites de x_j . (Resulta muy deseable tener límites estrechos para las variables individuales con el fin de aumentar la eficiencia de un algoritmo de templado simulado, lo cual también sucede en el caso de los algoritmos genéticos que se explican en la siguiente sección.) Cuando $n = 2$, también se puede incluir una o dos restricciones funcionales lineales sobre las variables $\mathbf{x} = (x_1, \dots, x_n)$. En el ejemplo, se tiene

$$n = 1, \quad L_1 = 0, \quad U_1 = 31,$$

sin restricciones funcionales lineales.

Este procedimiento del IOR Tutorial diseña los detalles del algoritmo de templado simulado de los problemas de programación no lineal de este tipo como se describe a continuación:

1. **Solución de prueba inicial:** Se puede introducir cualquier solución factible, pero resulta útil introducir una que parezca ser buena. En ausencia de algunas pistas acerca de dónde podrían estar las buenas soluciones factibles, es razonable establecer cada variable x_j en el punto medio entre su límite inferior L_j y la cota superior U_j con el propósito de comenzar la búsqueda a la mitad de la región factible. (Por esta razón, $x = 15.5$ es una selección razonable de la solución de prueba inicial del ejemplo.)
2. **Estructura de vecindad:** Cualquier solución factible puede tomarse en cuenta para ser un vecino inmediato de la solución de prueba actual. Sin embargo, el método que se describe a continuación para seleccionar un vecino inmediato que se convierte en el candidato actual a ser la próxima solución de prueba, otorga preferencia a las soluciones factibles que están relativamente cerca de la solución de prueba actual. Aun así, permite la posibilidad de trasladarse a una parte diferente de la región factible para continuar la búsqueda.
3. **Selección aleatoria de un vecino inmediato:** Se establece

$$\sigma_j = \frac{U_j - L_j}{6}, \text{ para } j = 1, \dots, n.$$

Después, dada la solución de prueba actual $(x_1, \dots, x_n)$,

$$\text{se reescribe } x_j = x_j + N(0, \sigma_j), \text{ para } j = 1, \dots, n,$$

donde $N(0, \sigma_j)$ es una observación aleatoria de una *distribución normal* con media cero y desviación estándar σ_j . Si este procedimiento no genera una solución factible, entonces se repite este procedimiento (se comienza de nuevo a partir de la solución de prueba actual) las veces que sea necesario para obtener una solución factible.

4. **Programa de temperatura:** Igual que en el caso de los problemas del agente viajero, se realizan cinco iteraciones a cada uno de los cinco valores de T (T_1, T_2, T_3, T_4, T_5), donde

$$\begin{aligned} T_1 &= 0.2Z_c \text{ cuando } Z_c \text{ es el valor de la función objetivo de la solución de prueba inicial,} \\ T_2 &= 0.5T_1, \\ T_3 &= 0.5T_2, \\ T_4 &= 0.5T_3, \\ T_5 &= 0.5T_4, \end{aligned}$$

La razón para establecer $\sigma_j = (U_j - L_j)/6$ al seleccionar un vecino inmediato es que cuando la variable x_j está en el punto medio entre L_j y U_j , cualquier nuevo valor factible de la variable está dentro de tres desviaciones estándar del valor actual. Esto proporciona una probabilidad significativa de que el nuevo valor se mueva hacia uno de sus límites incluso cuando hay mucha mayor probabilidad de que el nuevo valor esté relativamente cerca del valor actual. Existen varios métodos para generar una observación aleatoria $N(0, \sigma_j)$ a partir de una distribución normal (como se explicará de manera breve en la sección 20.4). Por ejemplo, la función de Excel NORMINV(RAND(),0, σ_j) genera una observación aleatoria de este tipo. Para los problemas adicionales se presenta una forma directa de generar las observaciones aleatorias que se requieran. Obtenga un número aleatorio r y después use la tabla normal del apéndice 5 para encontrar el valor de $N(0, \sigma_j)$ tal que $P\{X \leq N(0, \sigma_j)\} = r$ cuando X es una variable aleatoria normal con media 0 y desviación estándar σ_j .

Para ilustrar la forma en que el algoritmo diseñado de esta forma se aplicaría al ejemplo, se comenzará con $x = 15.5$ como la solución de prueba inicial. Así,

$$Z_c = f(15.5) = 3\,741\,121 \quad \text{y} \quad T_1 = 0.2Z_c = 748\,224.$$

Como

$$\sigma = \frac{U - L}{6} = \frac{31 - 0}{6} = 5.167,$$

el siguiente paso es generar una observación aleatoria $N(0, 5.167)$ a partir de una distribución normal con media cero y esta desviación estándar. Para hacer esto, primero se obtiene un número aleatorio, que resulta ser 0.0735. Si se consulta la tabla normal en el apéndice 5, $P\{\text{normal estándar} \leq -1.45\} = 0.0735$, de manera que $N(0, 5.167) = -1.45(5.167) = -7.5$. El candidato actual a ser la siguiente solución de prueba se obtiene al reescribir x como

$$\begin{aligned} x &= 15.5 + N(0, 5.167) = 15.5 - 7.5 \\ &= 8, \end{aligned}$$

así que

$$Z_n = f(x) = 3\,055\,616.$$

Debido a que

$$\frac{Z_n - Z_c}{T} = \frac{3\,055\,616 - 3\,741\,121}{748\,224} = -0.916$$

la probabilidad de aceptar $x = 8$ como la próxima solución de prueba es

$$\text{Prob}\{\text{aceptación}\} = e^{-0.916} = 0.400.$$

Por tanto, $x = 8$ se aceptará sólo si el número aleatorio correspondiente entre 0 y 1 resulta ser menor que 0.400. En consecuencia, es bastante probable que $x = 8$ sea rechazada. (En algunas iteraciones posteriores cuando T es mucho más pequeña, $x = 8$ tendría casi la certeza de ser rechazada.) Esta perspectiva resulta afortunada puesto que la figura 13.1 revela que la búsqueda debería enfocarse en la parte de la región factible entre $x = 10$ y $x = 30$ con el fin de comenzar a escalar hacia la cumbre más alta.

En la tabla 13.6 se proporcionan los resultados que se obtuvieron al utilizar el IOR Tutorial para aplicar totalmente el algoritmo de templado simulado a este problema de programación no

■ **TABLA 13.6** Aplicación del algoritmo de templado simulado del IOR Tutorial al ejemplo de programación no lineal

Iteración	T	Solución de prueba obtenida	$f(x)$
0		$x = 15.5$	3 741 121.0
1	748 224	$x = 17.557$	4 167 533.956
2	748 224	$x = 14.832$	3 590 466.203
3	748 224	$x = 17.681$	4 188 641.364
4	748 224	$x = 16.662$	3 995 966.078
5	748 224	$x = 18.444$	4 299 788.258
6	374 112	$x = 19.445$	4 386 985.033
7	374 112	$x = 21.437$	4 302 136.329
8	374 112	$x = 18.642$	4 322 687.873
9	374 112	$x = 22.432$	4 113 901.493
10	374 112	$x = 21.081$	4 345 233.403
11	187 056	$x = 20.383$	4 393 306.255
12	187 056	$x = 21.216$	4 330 358.125
13	187 056	$x = 21.354$	4 313 392.276
14	187 056	$x = 20.795$	4 370 624.01
15	187 056	$x = 18.895$	4 348 060.727
16	93 528	$x = 21.714$	4 259 787.734
17	93 528	$x = 19.463$	4 387 360.1
18	93 528	$x = 20.389$	4 393 076.988
19	93 528	$x = 19.83$	4 398 710.575
20	93 528	$x = 20.68$	4 378 591.085
21	46 764	$x = 20.031$	4 399 955.913 ← Máximo
22	46 764	$x = 20.184$	4 398 462.299
23	46 764	$x = 19.9$	4 399 551.462
24	46 764	$x = 19.677$	4 395 385.618
25	46 764	$x = 19.377$	4 383 048.039

lineal. Observe que las soluciones de prueba que se obtuvieron varían con bastante amplitud dentro de la región factible durante las primeras iteraciones, pero después comienzan a aproximarse a la cumbre más alta de una manera más coherente durante las últimas iteraciones cuando T se reduce a valores mucho más pequeños. Por tanto, de las 25 iteraciones, la mejor solución de prueba de $x = 20.031$ (en comparación con la solución óptima de $x = 20$) no se obtuvo hasta la iteración 21.

Una vez más, el lector puede encontrar interesante aplicar este software al mismo problema por sí mismo para ver qué se obtiene con las nuevas secuencias de números aleatorios y observaciones aleatorias a partir de distribuciones normales. (En el problema 13.3-6 se pide hacerlo varias veces.)

■ 13.4 ALGORITMOS GENÉTICOS

Los algoritmos genéticos proporcionan un tercer tipo de metaheurística que es muy diferente de las primeras dos. Este tipo tiende a ser muy eficaz para explorar diversas partes de la región factible y evolucionar de manera gradual hacia las mejores soluciones factibles.

Después de introducir los conceptos básicos de este tipo de metaheurística, se aplicará un algoritmo genético básico al mismo ejemplo de programación no lineal que se consideró en la sección anterior con la restricción adicional de que la variable se limita a valores enteros. Después se aplicará este enfoque al mismo ejemplo del problema del agente viajero que se consideró en cada una de las secciones precedentes.

Conceptos básicos

De la misma forma que el templado simulado se basa en una analogía con un fenómeno natural (el proceso físico de templado), los algoritmos genéticos están muy influidos por otra forma de un fenómeno natural. En este caso, la analogía es con la *teoría biológica de la evolución* formulada por Charles Darwin a mediados del siglo XIX. Cada especie de plantas y animales muestra grandes

variaciones individuales. Darwin observó que aquellos individuos con variaciones que significan una ventaja de supervivencia a través de una mejoría en la adaptación al entorno tienen una posibilidad mayor de sobrevivir en la siguiente generación. Este fenómeno se conoce desde entonces como la *supervivencia del más apto*.

El moderno campo de la genética proporciona una mejor explicación de este proceso de evolución y de la *selección natural* involucrada en la supervivencia del más apto. En algunas especies que se reproducen por vía sexual, cada descendiente hereda algunos de los *cromosomas* de cada uno de los dos padres, donde los *genes* de los cromosomas determinan las características individuales del hijo. Un hijo que hereda las mejores características de los padres tiene una mayor posibilidad de sobrevivir en su adultez y se convierte en un padre que pasa algunas de estas características a la siguiente generación. La población tiende a mejorar de manera lenta a través del tiempo por medio de este proceso. Un segundo factor que contribuye a este proceso es una tasa de mutación aleatoria y de bajo nivel en el DNA de los cromosomas. En este contexto, de vez en cuando ocurre una *mutación* que cambia las características de un cromosoma que un hijo hereda de un padre. Aunque la mayoría de las mutaciones no tiene ningún efecto o son desventajosas, algunas proporcionan mejoras deseables. Los hijos con mutaciones deseables tienen una posibilidad un poco mayor de sobrevivir para así contribuir a la reserva futura de genes de la especie.

Estas ideas se transfieren hacia los problemas de optimización en una forma bastante natural. Las soluciones factibles de un problema específico corresponden a los miembros de una especie particular, donde la aptitud de cada miembro ahora se mide por el valor de la función objetivo. En lugar de procesar una sola solución de prueba a la vez (como sucede con las formas básicas de la búsqueda tabú y el templado simulado), ahora se trabaja con una *población* completa de soluciones de prueba.¹ En el caso de cada iteración (generación) de un algoritmo genético, la **población** actual consiste en el conjunto de soluciones de prueba que en la actualidad están bajo consideración. Estas soluciones de prueba se entienden como los miembros vivos de la especie. Algunos de los miembros más jóvenes de la población (en especial los miembros más aptos) sobreviven en la adultez y se convierten en **padres** (aparejados de manera aleatoria) que después tienen **hijos** (nuevas soluciones de prueba) que tienen algunas de las características (genes) de ambos padres. Como los miembros más aptos de la población tienen una mayor probabilidad de convertirse en padres que los otros, a medida que avanza, un algoritmo genético tiende a generar *poblaciones mejoradas* de soluciones de prueba. De vez en cuando ocurren **mutaciones**, de forma que los hijos también pueden adquirir características (algunas veces deseables) que no posee ninguno de los padres. Este fenómeno ayuda a los algoritmos genéticos a explorar una parte de la región factible, quizá mejor que la que se consideró antes. Por último, la supervivencia del más apto tiende a conducir al algoritmo genético hacia una solución de prueba (la mejor de todas las consideradas) que al menos es cercana a la óptima.

Aunque la analogía con el proceso de la evolución biológica define la esencia de cualquier algoritmo genético, no es necesario adherirse de manera rígida a esta analogía en cada detalle. Por ejemplo, algunos algoritmos genéticos (incluido el que se describe a continuación) permiten que la misma solución de prueba sea un padre en forma repetida a través de múltiples generaciones. De modo que la analogía debe ser sólo un punto de partida para definir los detalles del algoritmo que se ajuste mejor al problema bajo consideración.

A continuación se presenta un esquema bastante típico de un algoritmo genético que se empleará para los dos ejemplos.

Esquema de un algoritmo genético básico

Paso inicial. Comience con una población de soluciones de prueba factibles, quizá generándolas en forma aleatoria. Evalúe la *aptitud* (el valor de la función objetivo) de cada miembro de esta población actual.

Iteración. Use un proceso aleatorio que esté sesgado hacia los miembros más aptos de la población actual para seleccionar algunos de ellos (un número par) para convertirse en padres. Apareje

¹ Una de las estrategias de intensificación de la búsqueda tabú también mantiene una población de las mejores soluciones. La población se utiliza para crear rutas de vinculación entre sus miembros y para reiniciar la búsqueda a lo largo de estas rutas.

los padres de manera aleatoria para que de ellos nazcan dos hijos (nuevas soluciones de prueba *factibles*) cuyas características (genes) sean una mezcla aleatoria de las características de los padres, excepto por mutaciones ocasionales. (Cuando la mezcla aleatoria de características y algunas mutaciones generan una solución *no factible*, ésta se considera un *aborto*, y el proceso de intentar un nacimiento se repite hasta que nace un hijo que corresponde a una solución *factible*.) Conserve los hijos y una cantidad suficiente de los mejores miembros de la población actual para formar la nueva población del mismo tamaño de la próxima iteración. (Descarte los otros miembros de la población actual.) Evalúe la aptitud de cada nuevo miembro (hijo) de la población.

Regla de detención. Utilice alguna regla de detención, como un número fijo de iteraciones, una cantidad de tiempo de CPU fija o un número fijo de iteraciones consecutivas sin ninguna mejora a la mejor solución de prueba que haya encontrado hasta ese momento. Use la mejor solución de prueba que encontró en cualquier iteración como la solución final.

Antes de que este algoritmo se pueda implantar, es necesario contestar las siguientes preguntas:

1. ¿Cuál debe ser el tamaño de la población?
2. ¿Cómo deben seleccionarse los miembros de la población actual para convertirse en padres?
3. ¿Cuáles deben ser las características de los hijos derivadas de las características de los padres?
4. ¿Cómo deben insertarse las mutaciones en las características de los hijos?
5. ¿Cuál es la regla de detención que debe usarse?

Las respuestas a estas preguntas dependen en gran medida de la estructura del problema específico que se trata de resolver. En el área de metaheurísticas del IOR Tutorial se incluyen dos versiones del algoritmo. Una de ellas sirve para resolver problemas muy pequeños de programación no lineal entera como el ejemplo que se considera a continuación. La otra es para manejar problemas pequeños del agente viajero. Ambas versiones contestan algunas de las preguntas del mismo modo, como se describe en seguida.

1. **Tamaño de población:** Diez. (Este tamaño es razonable en los pequeños problemas para los que está diseñado este software, pero para problemas más grandes se usan poblaciones mucho más extensas.)
2. **Selección de padres:** De entre los cinco miembros más aptos de la población (de acuerdo con el valor de la función objetivo), seleccione cuatro de manera aleatoria para convertirse en padres. De entre los cinco miembros menos aptos, seleccione dos en forma aleatoria para volverse padres. Apareje los seis padres de manera aleatoria para formar tres parejas.
3. **Transferencia de características (genes) de padres a hijos:** En gran medida, este proceso depende del problema y por ende es diferente para las dos versiones del algoritmo del software, como se describe después para los dos ejemplos.
4. **Tasa de mutación:** La probabilidad de que una característica heredada de un hijo mute hacia una característica opuesta se establece en el software como 0.1. (En el caso de problemas más grandes es común utilizar tasas de mutación mucho más pequeñas.)
5. **Regla de detención:** El algoritmo se detiene después de cinco iteraciones sin ninguna mejora a la mejor solución de prueba que se haya encontrado hasta ese momento.

Ahora es posible aplicar el algoritmo a los dos ejemplos.

Versión entera del ejemplo de programación no lineal

Considere de nuevo el pequeño problema de programación no lineal que se introdujo en la sección 13.1 (vea la figura 13.1) y que después se resolvió con un algoritmo de templado simulado al final de la sección anterior. Ahora, a este ejemplo se le agrega la restricción de que la única variable x del problema debe tener un valor entero. Como el problema ya tiene la restricción de que $0 \leq x \leq 31$, lo cual significa que el problema tiene 32 soluciones factibles, $x = 0, 1, 2, \dots, 31$. (La existencia de estos límites es muy importante para un algoritmo genético, puesto que reduce el espacio de búsqueda a la región relevante.) Por lo tanto, el ejemplo se ha convertido en un problema de programación no lineal *entera*.

Cuando se aplica un algoritmo genético, con frecuencia se utilizan *cadena de dígitos binarios* para representar las soluciones del problema. Tal *codificación* de las soluciones es particularmente conveniente para los diferentes pasos de un algoritmo genético, incluido el proceso de nacimiento de los hijos. Esta codificación es fácil de hacer para este problema en particular, porque simplemente se puede escribir cada valor de x en base 2. Como el máximo valor factible de x es 31, sólo se requieren cinco dígitos binarios para escribir cualquier valor factible. Siempre se incluirán los cinco dígitos binarios aunque el primero o los primeros dígitos sean ceros. Así, por ejemplo,

$x = 3$ es 00011 en base 2,
 $x = 10$ es 01010 en base 2,
 $x = 25$ es 11001 en base 2.

Cada uno de los cinco dígitos binarios se conoce como uno de los **genes** de la solución, donde los dos valores posibles del dígito binario describen cuál de las dos posibles características se conserva en ese gen para ayudar a formar la composición genética global. Cuando los dos padres tienen la misma característica, éste se pasará a cada hijo (excepto cuando ocurre una mutación). Sin embargo, cuando los dos padres tienen características opuestas del mismo gen, la característica que heredará el hijo se vuelve aleatoria.

Por ejemplo, suponga que los dos padres son

P1: 00011 y
 P2: 01010.

Como el primero, tercero y cuarto dígitos coinciden, en forma automática los hijos serán (salvo mutaciones)

C1: 0x01x y
 C2: 0x01x,

donde x indica que este dígito particular todavía no se conoce. Para identificar los dígitos desconocidos se utilizan números aleatorios, donde una correspondencia natural es

0.0000-0.4999 corresponde a que el dígito sea 0.
 0.5000-0.9999 corresponde a que el dígito sea 1.

Por ejemplo, suponga que los siguientes cuatro números aleatorios generados son 0.7265, 0.5190, 0.0402 y 0.3639, de manera que los dos dígitos desconocidos del primer hijo son 1 y los dos dígitos desconocidos del segundo hijo son 0. En este caso los hijos serán (salvo mutaciones)

C1: 01011 y
 C2: 00010.

Este método particular para generar hijos a partir de los padres se conoce como *cruce uniforme*. Quizá es el más intuitivo de los diferentes métodos alternativos que se han propuesto.

Ahora es necesario considerar la posibilidad de mutaciones que pudieran afectar la conformación genética del hijo.

Como la probabilidad de una mutación en algún gen (que cambia el valor de un dígito binario al valor opuesto) se ha establecido en 0.1 para este algoritmo, se pueden utilizar de nuevo números aleatorios, donde

0.0000-0.0999 corresponde a una mutación,
 0.1000-0.9999 corresponde a que no hay mutación.

Por ejemplo, suponga que en los siguientes 10 números aleatorios generados, sólo el octavo es menor a 0.1000. Esto indica que no ocurre mutación en el primer hijo, pero el tercer gen (dígito) del segundo hijo invierte su valor. Por tanto, la conclusión final es que los dos hijos son

C1: 01011 y
 C2: 00110.

Si se regresa a la base 10, los dos padres corresponden a las soluciones $x = 3$ y $x = 10$, mientras que sus hijos serían (salvo mutaciones) $x = 11$ y $x = 2$. Sin embargo, debido a la mutación, los hijos se convierten en $x = 11$ y $x = 6$.

En este ejemplo particular, cualquier valor entero de x tal que $0 \leq x \leq 31$ (en base 10) es una solución factible, por lo que cada número de 5 dígitos en base 2 también es una solución factible. Por tanto, el proceso anterior de crear hijos nunca resulta en un *aborto* (una solución factible). Sin embargo, si el límite superior sobre x fuera, por ejemplo, $x \leq 25$, entonces ocurrirán abortos de manera ocasional. Siempre que sucede un aborto se descarta la solución y se repite el proceso completo de creación de un hijo hasta que se obtiene una solución factible.

Este ejemplo incluye sólo una variable. Para problemas de programación no lineal con múltiples variables, cada miembro de la población usaría de nuevo la base dos para mostrar el valor de cada variable. Entonces, el proceso anterior para generar hijos a partir de los padres se realizaría de la misma forma para una variable a la vez.

En la tabla 13.7 se muestra la aplicación completa del algoritmo a este ejemplo a través del paso inicial [parte *a*) de la tabla] y de la iteración 1 [parte *b*) de la tabla]. En el paso de inicio, cada uno de los miembros de la población inicial se generó mediante cinco números aleatorios y la utilización de la correspondencia entre un número aleatorio y un dígito binario, que se dio con anterioridad para obtener los cinco dígitos binarios a la vez. El valor correspondiente de x en base 10 se inserta en la función objetivo que se dio al inicio de la sección 13.1 para evaluar la aptitud de ese miembro de la población.

Los cinco números de la población inicial que tienen el grado más alto de aptitud (en orden) son los miembros 10, 8, 4, 1 y 7. Para elegir en forma aleatoria cuatro de estos miembros para convertirse en padres, se usa un número aleatorio para seleccionar un miembro que será rechazado, donde 0.0000-0.1999 significa la expulsión del primer miembro de la lista (miembro 10), 0.2000-0.3999 corresponde a expulsar el segundo miembro, y así sucesivamente. En este caso, el número aleatorio fue 0.9665 (miembro 7), por lo que el quinto miembro de la lista no se convierte en padre.

De entre los cinco miembros menos aptos de la población inicial (miembros 2, 1, 6, 5 y 9), ahora se usan números aleatorios para seleccionar cuáles dos de estos miembros se convertirán en padres. En este caso los números aleatorios fueron 0.5634 y 0.1270. Con referencia al primer número aleatorio, 0.0000-0.1999 corresponde seleccionar el primer número de la lista (miembro 2), 0.2000-0.3999 corresponde a seleccionar el segundo miembro, y así sucesivamente, por lo que

■ **TABLA 13.7** Aplicación del algoritmo genético al ejemplo de programación no lineal entera durante *a*) el paso inicial y *b*) la iteración 1

Miembro		Población inicial		Valor de x	Aptitud
a)	1	0 1 1 1 1		15	3 628 125
	2	0 0 1 0 0		4	3 234 688
	3	0 1 0 0 0		8	3 055 616
	4	1 0 1 1 1		23	3 962 091
	5	0 1 0 1 0		10	2 950 000
	6	0 1 0 0 1		9	2 978 613
	7	0 0 1 0 1		5	3 303 125
	8	1 0 0 1 0		18	4 239 216
	9	1 1 1 1 0		30	1 350 000
	10	1 0 1 0 1		21	4 353 187
Miembro		Padres	Hijo	Valor de x	Aptitud
10		1 0 1 0 1	0 0 1 0 1	5	3 303 125
2		0 0 1 0 0	1 0 0 0 1	17	4 064 259
b)	8	1 0 0 1 0	1 0 0 1 1	19	4 357 164
	4	1 0 1 1 1	1 0 1 0 0	20	4 400 000
1		0 1 1 1 1	0 1 0 1 1	11	2 980 637
6		0 1 0 0 1	0 1 1 1 1	15	3 628 125

en este caso el miembro seleccionado es el tercero (miembro 6). Como ahora restan sólo cuatro números (2, 1, 5 y 9) para seleccionar el último padre, los intervalos correspondientes al segundo número aleatorio son 0.0000-0.2499, 0.2500-0.4999, 0.5000-0.7499 y 0.7500-0.9999. Como 0.1270 cae en el primero de estos intervalos se selecciona el primer miembro de la lista (miembro 2) para ser un padre.

El siguiente paso es aparejar los seis padres, esto es, los miembros 10, 8, 4, 1, 6 y 2. Se comenzará por usar un número aleatorio para determinar la pareja del primer miembro de la lista (miembro 10). El número aleatorio 0.8204 indicó que 10 debe estar aparejado con el quinto de los otros cinco padres de la lista (miembro 2). Para aparejar el siguiente miembro de la lista (miembro 8), el número aleatorio fue 0.0198, que se encuentra en el intervalo 0.0000-0.3333, por lo que el primero de los tres padres restantes (miembro 4) se elige para ser el compañero del miembro 8. Con esto restan dos padres (miembros 1 y 6) que se convierten en la última pareja.

En la parte *b*) de la tabla 13.7 se muestran los hijos reproducidos por estos padres por medio del proceso que se ilustró antes en esta subsección. Observe las mutaciones que sufrió el tercer gen del segundo hijo y el cuarto gen del cuarto hijo. Asimismo, el sexto hijo tiene un grado de aptitud relativamente alto. En realidad, para cada pareja, ambos hijos resultaron ser más aptos que uno de los padres. Esto no sucede siempre, pero es bastante común. En el caso de la segunda pareja de padres, los dos hijos resultaron ser más aptos que ambos padres. De manera fortuita, los dos hijos ($x = 19$ y $x = 20$) en realidad son superiores a *cualquiera* de los miembros de la población anterior que se presentó en la parte *a*) de la tabla. Para formar la nueva población de la próxima iteración, los seis hijos se conservan junto con los cuatro miembros más aptos de la población anterior (miembros 10, 8, 4 y 1).

Las iteraciones siguientes se realizan de un modo similar. Como ya se sabe con base en la explicación de la sección 13.1 (vea la figura 13.1) que $x = 20$ (la mejor solución de prueba generada en la iteración 1) en realidad es la solución óptima de este ejemplo, las iteraciones subsecuentes no proporcionarían ninguna mejora. Por tanto, la regla de detención terminaría el algoritmo después de cinco iteraciones más y se obtendría $x = 20$ como la solución final.

En el IOR Tutorial se incluye una rutina para aplicar este mismo algoritmo genético a otros problemas muy pequeños de programación no lineal entera. (La forma y el tamaño de las restricciones son los mismos que los que se especificaron en la sección 13.3 para el caso de problemas de programación no lineal.)

El lector podría encontrar interesante la aplicación de esta rutina del IOR Tutorial a este mismo ejemplo. Debido a la aleatoriedad inherente del algoritmo, se obtienen resultados intermedios diferentes cada vez que éste se aplica. (En el problema 13.4-3 se pide aplicar el algoritmo a este ejemplo varias veces.)

Aunque éste fue un ejemplo discreto, los algoritmos genéticos también se pueden aplicar a problemas continuos, como un problema de programación no lineal sin la restricción de valores enteros. En este caso, el valor de una variable continua se representaría (o tendría una aproximación cercana) por medio de un número decimal en base 2. Por ejemplo, $x = 23\frac{5}{8}$ es 10111.10100 en base 2, y $x = 23.66$ es muy cercano a 10111.10101 en base 2. Todos los dígitos binarios de ambos lados del punto decimal se pueden tratar como se hizo antes para aparejar padres que producen hijos, etcétera.

Ejemplo del problema del agente viajero

En las secciones 13.2 y 13.3 se ilustró cómo se aplican los algoritmos de búsqueda tabú y de templado simulado al problema del agente viajero particular que se introdujo en la sección 13.1 (vea la figura 13.4). Ahora se verá cómo puede aplicarse el algoritmo genético a este mismo ejemplo.

En este caso, en lugar de utilizar dígitos binarios se continuará con la representación de cada solución (viaje) en la forma natural como una secuencia de ciudades visitadas. Por ejemplo, la primera solución que se considera en la sección 13.1 es el viaje por las ciudades en el siguiente orden: 1-2-3-4-5-6-7-1, donde la ciudad 1 es la población de residencia en la que el viaje debe iniciarse y terminar. Sin embargo, debe puntualizarse que los algoritmos genéticos para manejar los problemas del agente viajero con frecuencia utilizan otros métodos para *codificar* soluciones. En general, los métodos inteligentes para representar soluciones (a menudo con la utilización de cadenas de dígitos binarios) pueden hacer más fácil la generación de hijos, la creación de mutaciones, la conservación de factibilidad, etc., de un modo natural. El desarrollo de un *esquema de*

codificación apropiado es una parte clave de la elaboración de un algoritmo genético eficaz para cualquier aplicación.

Una complicación con este ejemplo particular es que, en cierto sentido, es muy fácil. Debido al número bastante limitado de ligaduras entre los pares de ciudades de la figura 13.4, este problema apenas alcanza 10 soluciones factibles distintas si se descartan los viajes que son sólo un viaje considerado antes pero en la dirección inversa. Por tanto, no es posible tener una población inicial con 10 distintas soluciones de prueba tales que los seis padres resultantes reproduzcan hijos diferentes que también sean distintos de los miembros de la población inicial (incluidos los padres).

Por fortuna, un algoritmo genético puede aun operar en forma razonable cuando existe una tasa modesta de duplicación de las soluciones de prueba en una población o en dos poblaciones consecutivas. Por ejemplo, aun cuando los dos padres de una pareja sean idénticos, es posible que los hijos sean diferentes a los padres debido a las mutaciones.

El algoritmo genético para manejar problemas del agente viajero del IOR Tutorial no hace nada para evitar la duplicación de las soluciones de prueba consideradas. Cada una de las 10 soluciones de prueba de la población inicial se genera de la manera siguiente. A partir de la ciudad de residencia se usan números aleatorios para seleccionar la siguiente ciudad de entre aquellas que tienen una ligadura con la ciudad inicial (2, 3 y 7 en la figura 13.4). Después se usan números aleatorios para seleccionar la tercera ciudad de entre las ciudades restantes que tienen una ligadura con la segunda ciudad. Este proceso se continúa hasta que todas las ciudades se incluyen una vez en el viaje (más un regreso a la ciudad de residencia a partir de la última ciudad) o se llega a un final anticipado porque no existen ligaduras de la ciudad actual con ninguna de las ciudades que aún no se han visitado. En el último caso se reinicia el proceso completo para generar una solución de prueba, desde el principio y con números aleatorios nuevos.

También se usan números aleatorios para reproducir hijos a partir de una pareja de padres. Para ilustrar este proceso, considere el siguiente par de padres.

P1: 1-2-3-4-5-6-7-1

P2: 1-2-4-6-5-7-3-1

Conforme se describa el proceso de generación de hijos a partir de estos padres, se presentará un resumen de estos resultados en la tabla 13.8 para ayudarlo a seguir el avance.

Si por el momento no se toma en cuenta la posibilidad de mutaciones, a continuación se describe la idea fundamental de cómo generar un hijo.

Enlaces heredados: Los genes corresponden a los enlaces en un viaje. Entonces cada uno de los genes (ligaduras) heredado por un hijo debe provenir de uno de los padres (o de ambos). (La excepción descrita en el próximo párrafo es que un padre también puede pasar a su hijo un subviaje inverso.) Debido a que son heredados, estos enlaces se seleccionan aleatoriamente uno a la vez hasta que se genera un viaje completo (el hijo).

Para comenzar este proceso con los padres mencionados antes, como un viaje debe comenzar en la ciudad 1, la ligadura inicial de un hijo debe provenir de una de las ligaduras de los padres que

■ **TABLA 13.8** Ejemplo del proceso de generación de un hijo para el caso del agente viajero

Padre P1:	1-2-3-4-5-6-7-1		
Padre P2:	1-2-4-6-5-7-3-1		
Ligadura	Opciones	Selección aleatoria	Viaje
1	1-2, 1-7, 1-2, 1-3	1-2	1-2
2	2-3, 2-4	2-4	1-2-4
3	4-3, 4-5, 4-6	4-3	1-2-4-3
4	3-5*, 3-7	3-5*	1-2-4-3-5
5	5-6, 5-6, 5-7	5-6	1-2-4-3-5-6
6	6-7	6-7	1-2-4-3-5-6-7
7	7-1	7-1	1-2-4-3-5-6-7-1

*Ligadura que termina un subviaje inverso.

conectan la ciudad 1 con otra ciudad. En el caso del padre P1, éstas son las ligaduras 1-2 y 1-7. (La ligadura 1-7 califica puesto que es equivalente tomar el viaje en cualquier dirección.) En el del padre P2, las ligaduras correspondientes son 1-2 (de nuevo) y 1-3. El hecho de que ambos padres tengan la ligadura 1-2 duplica la probabilidad de que ésta sea heredada por un hijo. Por lo tanto, cuando se usa un número aleatorio para determinar cuál ligadura heredará el hijo, el intervalo 0.0000-0.4999 (o cualquier intervalo de este tamaño) corresponde a heredar la ligadura 1-2 mientras que los intervalos 0.50000-0.7499 y 0.7500-0.9999 corresponderían a la elección de la ligadura 1-7 y la ligadura 1-3, respectivamente. Suponga que se elige 1-2 como se muestra en el primer renglón de la tabla 13.8. Después de 1-2, un padre utiliza en seguida la ligadura 2-3 mientras que el otro usa 2-4. Por tanto, al generar el hijo, se debe hacer una selección aleatoria entre estas dos opciones. Suponga que se selecciona 2-4 (vea el segundo renglón de la tabla 13.8). Ahora existen tres opciones para la ligadura que sigue a 1-2-4 porque el primer padre usa dos ligaduras (4-3 y 4-5) para conectar a la ciudad 4 en su viaje y el segundo padre usa la ligadura 4-6 (la ligadura 4-2 se pasa por alto porque la ciudad 2 ya está en el viaje del hijo). Al seleccionar de manera aleatoria una de estas opciones, suponga que se elige 4-3 para formar 1-2-4-3 como el inicio del viaje del hijo hasta ahora, como se muestra en el tercer renglón de la tabla 13.8.

Ahora se llega a una característica adicional de este proceso para generar el viaje de un hijo, a saber, el uso de un *subviaje inverso* del padre.

Subviaje inverso heredado: Otra posibilidad de un enlace heredado por un hijo es un enlace que es necesario para completar un subviaje inverso al del viaje que el hijo está haciendo en una parte de un viaje del padre.

Para ilustrar cómo puede surgir esta posibilidad, observe que la siguiente ciudad después de 1-2-4-3 debe ser una de las ciudades que no se ha visitado (ciudad 5, 6 o 7). Pero el primer padre no tiene ninguna ligadura de la ciudad 3 a alguna de estas otras ciudades. La razón es que el hijo utiliza un subviaje inverso (al invertir 3-4) del viaje de este padre, 1-2-3-4-5-6-7-1. Para completar este subviaje inverso se requiere agregar la ligadura 3-5, con lo cual ésta se convierte en una de las opciones para la siguiente ligadura del viaje del hijo. La otra opción es la ligadura 3-7 que proporciona el segundo padre (la ligadura 3-1 no se considera una opción porque la ciudad 1 debe estar hasta el final del viaje). Una de estas dos opciones se selecciona de manera aleatoria. Suponga que la elección es la ligadura 3-5, lo cual proporciona el viaje del hijo hasta el momento, 1-2-4-3-5, como se muestra en el cuarto renglón de la tabla 13.8.

Para continuar este viaje, las opciones de la siguiente ligadura son 5-6 (que proporcionan los dos padres) y 5-7 (que proporciona el segundo padre). Suponga que la elección aleatoria entre 5-6, 5-6 y 5-7 es 5-6, por lo que el viaje hasta ahora es 1-2-4-3-5-6. (Vea la quinta hilera de la tabla 13.8.) Como la única ciudad que aún no se visita es la 7, se agrega de manera automática la ligadura 6-7 seguida por la ligadura 7-1 para regresar a la ciudad de residencia. De esta forma, el viaje completo del hijo es

C1: 1-2-4-3-5-6-7-1

En la figura 13.5 de la sección 13.1 se observa qué tanto se parece este hijo al primer padre, puesto que la única diferencia es el subviaje inverso que se obtuvo al invertir 3-4 en el viaje del padre.

Si se hubiera elegido la ligadura 5-7 para agregarse a 1-2-4-3-5, el viaje se hubiera completado de manera automática como 1-2-4-3-5-7-6-1. Sin embargo, no existe una ligadura 6-1 (vea la figura 13.4), por lo que se llega a un final anticipado en la ciudad 6. Cuando esto sucede, ocurre un *aborto* y es necesario reiniciar todo el proceso desde el principio y con números aleatorios nuevos hasta que se obtenga un hijo con un viaje completo. Después se repite este proceso para obtener el segundo hijo.

Ahora es necesario agregar una característica más —la posibilidad de mutaciones— para completar la descripción del proceso de generación de hijos.

Mutaciones de enlaces heredados: Siempre que un enlace particular sea heredado a partir del padre de un hijo, existe una reducida probabilidad de que ocurra una mutación que rechace ese enlace y, en lugar de ello, seleccione aleatoriamente uno de los demás enlaces de la ciudad actual a otra ciudad que no esté en el viaje, sin considerar si dicho enlace es usado por cualquiera de los padres.

El algoritmo genético para los problemas del agente viajero implantado en su tutorial IOR utiliza una probabilidad de 0.1 de que ocurra una mutación cada vez que sea necesario seleccionar el enlace siguiente en el viaje del hijo. Así, siempre que el número aleatorio correspondiente sea menor a 0.1000, la elección de la ligadura que se realizó de la forma normal que se describió con anterioridad se rechaza (si existe alguna otra elección posible). En lugar de ello, todas las otras ligaduras que parten de la ciudad actual hacia una ciudad que aún no está en el viaje (incluso ligaduras no proporcionadas por los padres) se identifican, y una de ellas se selecciona en forma aleatoria para ser la siguiente ligadura del viaje. Por ejemplo, suponga que ocurre una mutación al generar la primera ligadura del hijo. A pesar de que se había elegido 1-2 de manera aleatoria como la primera ligadura, ahora ésta se debería rechazar debido a la mutación. En razón de que la ciudad 1 también tiene ligas con las ciudades 3 y 7 (vea la figura 13.4), cualquiera de las ligaduras 1-3 o 1-7 se selecciona en forma aleatoria para ser el primer viaje. (Como los padres terminan sus viajes con el uso de una u otra de estas ligaduras, este caso se puede entender como el inicio del viaje del hijo invirtiendo la dirección de los viajes de uno de los padres.)

Ahora es posible escribir un esquema del procedimiento general para generar un hijo a partir de una pareja de padres.

Procedimiento para generar un hijo

1. **Paso inicial:** Para comenzar, designe la ciudad de residencia como la *ciudad actual*.
2. **Opciones de la siguiente ligadura:** Identifique todas las ligaduras que van de la ciudad actual a otra ciudad, que todavía no están en el viaje del hijo y que sean utilizadas por los padres en cualquier dirección. Además, agregue cualquier ligadura que se necesite para completar un subviaje inverso que realiza el hijo en una parte del viaje del padre.
3. **Selección de la siguiente ligadura:** Use un número aleatorio para seleccionar de manera aleatoria una de las opciones identificadas en el paso 2.
4. **Verificación de una mutación:** Si el siguiente número aleatorio es menor que 0.1000, ocurre una mutación y la ligadura seleccionada en el paso 3 se rechaza (a menos que no exista otra ligadura desde la ciudad actual hasta otra ciudad que no se encuentre en el viaje). Si la ligadura se rechaza, identifique todas las otras ligaduras que parten de la ciudad actual y van a otra ciudad que no esté en el viaje del hijo (lo cual incluye las ligaduras no utilizadas por ningún padre). Use un número aleatorio para seleccionar de manera aleatoria una de estas otras ligaduras.
5. **Continuación:** Agregue la ligadura que seleccionó en el paso 3 (si no ocurre mutación) o en el paso 4 (si hay mutación) al final del viaje incompleto actual del hijo y reasigne la ciudad al final de esta ligadura como la *ciudad actual*. Si aún queda más de una ciudad no incluida en el viaje (además del regreso a la ciudad de residencia), regrese a los pasos 2-4 para seleccionar la siguiente ligadura. De otra forma, vaya al paso 6.
6. **Terminación:** Cuando sólo reste una ciudad sin incluir en el viaje del hijo, agregue la ligadura desde la ciudad actual hasta esta ciudad restante. Después agregue la ligadura que va de la última ciudad a la ciudad de residencia para completar el viaje del hijo. Sin embargo, si la ligadura necesaria no existe ocurre un aborto y el procedimiento debe reiniciarse de nuevo desde el paso 1.

Este procedimiento se aplica para cada par de padres para obtener cada uno de los dos hijos.

El algoritmo genético para problemas del agente viajero del IOR Tutorial incorpora este procedimiento para generar hijos como parte de todos los algoritmos descritos al comienzo de esta sección. En la tabla 13.9 se muestran los resultados de la aplicación de este algoritmo al ejemplo durante el paso inicial y la primera iteración de todo el algoritmo. Debido a la aleatoriedad que se incorpora al algoritmo, sus resultados intermedios (y quizá también la mejor solución final) variarán cada vez que se corra el algoritmo hasta su culminación. (Para explorar más a fondo esta cuestión, en el problema 13.4-7 se pide utilizar el IOR Tutorial para aplicar el algoritmo completo a este ejemplo varias veces.)

El hecho de que el ejemplo tenga sólo un número relativamente pequeño de soluciones factibles distintas se refleja en los resultados que se muestran en la tabla 13.9. Los miembros 1, 4, 6 y 10 son idénticos, al igual que los miembros 2, 7 y 9 (excepto porque el miembro 2 realiza su viaje en la dirección inversa). Por lo tanto, la generación aleatoria de 10 miembros de la población inicial

■ **TABLA 13.9** Aplicación del algoritmo genético del IOR Tutorial al ejemplo del problema del agente viajero durante a) el paso inicial y b) la iteración 1

Miembro		Población inicial		Distancia		
a)	1	1-2-4-6-5-3-7-1		64		
	2	1-2-3-5-4-6-7-1		65		
	3	1-7-5-6-4-2-3-1		65		
	4	1-2-4-6-5-3-7-1		64		
	5	1-3-7-5-6-4-2-1		66		
	6	1-2-4-6-5-3-7-1		64		
	7	1-7-6-4-5-3-2-1		65		
	8	1-3-7-6-5-4-2-1		69		
	9	1-7-6-4-5-3-2-1		65		
	10	1-2-4-6-5-3-7-1		64		
Miembro		Padres	Hijo	Miembro	Distancia	
1		1-2-4-6-5-3-7-1	1-2-4-5-6-7-3-1	11	69	
7		1-7-6-4-5-3-2-1	1-2-4-6-5-3-7-1	12	64	
b)	2		1-2-3-5-4-6-7-1	1-2-4-5-6-7-3-1	13	69
	6		1-2-4-6-5-3-7-1	1-7-6-4-5-3-2-1	14	65
4		1-2-4-6-5-3-7-1	1-2-4-6-5-3-7-1	15	64	
5		1-3-7-5-6-4-2-1	1-3-7-5-6-4-2-1	16	66	

resultó en sólo cinco distintas soluciones factibles. En forma similar, cuatro de los seis hijos generados (miembros 12, 14, 15 y 16) son idénticos a uno de sus padres (excepto porque el miembro 14 realiza el viaje en la dirección opuesta a su primer padre). Dos de los hijos (miembros 12 y 15) tienen una mejor aptitud (distancia más corta) que uno de sus padres, pero ninguno superó a ambos padres. Ninguno de estos hijos proporcionó una solución óptima (que tiene una distancia de 63). Esto ilustra el hecho de que, para algunos problemas, un algoritmo genético puede requerir muchas generaciones (iteraciones) antes de que el fenómeno de la supervivencia del más apto resulte en la creación de poblaciones significativamente superiores.

En la sección de Worked Examples del sitio en internet de este libro, se proporciona **otro ejemplo** de la aplicación de este algoritmo genético a un problema del agente viajero. Este problema tiene un número más grande de soluciones factibles que el ejemplo anterior, por lo cual hay una gran diversidad en la población inicial, los padres resultantes y sus hijos.

■ 13.5 CONCLUSIONES

Algunos problemas de optimización (incluidos diferentes problemas de optimización combinatoria) son tan complejos que resulta imposible resolverlos y encontrar una solución óptima con el tipo de algoritmos exactos que se presentaron en los capítulos anteriores. En tales casos, los métodos heurísticos se usan por lo general para buscar una buena solución factible (no necesariamente la óptima). Existen varias metaheurísticas que proporcionan una estructura general y directrices estratégicas para diseñar un método heurístico específico que se ajuste a un problema en particular. Una característica clave de estos procedimientos metaheurísticos es su capacidad de escapar de un óptimo local y realizar una búsqueda vigorosa en la región factible.

Este capítulo introdujo los tres tipos de metaheurísticas más prominentes. La *búsqueda tabú* se mueve desde la solución de prueba actual hasta la mejor solución de prueba vecina en cada iteración, procedimiento muy parecido a uno de mejora local, con la salvedad de que permite un movimiento sin mejora cuando no existen movimientos que mejoren la solución de prueba actual. También incorpora una memoria a corto plazo de la búsqueda pasada para incentivar el movimiento hacia nuevas áreas de la región factible en lugar de regresar a soluciones consideradas con anterior-

ridad. Además, puede utilizar estrategias de intensificación y diversificación basadas en la memoria a largo plazo para enfocar la búsqueda en continuaciones promisorias. El *templado simulado* también se desplaza desde la solución de prueba actual hasta una solución de prueba vecina en cada iteración mientras permite de manera ocasional movimientos sin mejora. Sin embargo, seleccione la solución de prueba vecina de manera aleatoria y después utilice la analogía con un proceso de templado físico para determinar si este vecino debe rechazarse como la próxima solución de prueba si no es tan bueno como la solución de prueba actual. El tercer tipo de metaheurística, los *algoritmos genéticos*, trabaja con una población completa de soluciones de prueba en cada iteración. Después utiliza la analogía con la teoría biológica de la evolución, sobre todo el concepto de la supervivencia del más apto, para descartar algunas de las soluciones de prueba (en especial las más pobres) y reemplazarlas con otras nuevas. Este proceso de reemplazo tiene pares de miembros sobrevivientes que transfieren algunas de sus características a pares de nuevos miembros como si fueran padres que reproducen hijos.

Con la intención de ser concretos, se describió el algoritmo básico de cada metaheurística y después se adaptó a dos tipos específicos de problemas (incluido el problema del agente viajero), mediante el uso de ejemplos simples. Sin embargo, los investigadores también han desarrollado muchas variaciones de cada algoritmo que son utilizadas por los practicantes para adaptarlas a las características de los problemas complejos a los que se enfrentan. Por ejemplo, se han propuesto docenas de variaciones del algoritmo genético básico para problemas del agente viajero (entre ellas diferentes procedimientos para generar hijos), y la investigación continúa para determinar cuál es la más eficaz. (Algunos de los mejores métodos para problemas del agente viajero usan estrategias especiales de “k-opt” y “cadena de expulsión” que se diseñan de manera cuidadosa para tomar ventaja de la estructura del problema.) Por lo tanto, las lecciones importantes de este capítulo son los conceptos básicos y la intuición incorporada en cada metaheurística más que los detalles de los algoritmos particulares que se presentaron.

Existen algunos otros tipos importantes de metaheurística además de los tres que se presentaron en este capítulo. Dentro de éstos, por ejemplo, se encuentran la optimización de la colonia de hormigas, la búsqueda esparcida y las redes neuronales artificiales. (Estos nombres sugestivos proporcionan una clave de la idea básica que está detrás de esta metaheurística.) La referencia seleccionada 4 ofrece una cobertura a fondo de esta metaheurística y de las tres que se presentaron aquí. (Michel Gendreau y Jean-Yves Potvin están preparando una segunda edición con el fin de actualizar esta importante referencia.)

En realidad, algunos algoritmos heurísticos son un híbrido de diferentes tipos de metaheurísticas que tratan de combinar sus mejores características. Por ejemplo, la búsqueda tabú a corto plazo (sin componente de diversificación) es muy eficiente para buscar óptimos locales pero no lo es tanto para explorar las diferentes áreas de la región factible para encontrar la parte que contiene el óptimo global, mientras que un algoritmo genético tiene las características opuestas. Por lo tanto, algunas veces se puede obtener un algoritmo mejorado si se comienza con un algoritmo genético para tratar de encontrar la cumbre más alta (cuando el objetivo es maximizar) y después se cambia a una búsqueda tabú básica para, por último, escalar hasta dicha cumbre. También existen otras metaheurísticas que son menos prominentes que las tres que se presentaron aquí, cuyas ideas básicas también podrían incorporarse en un algoritmo heurístico. La clave para diseñar un algoritmo heurístico eficaz es incorporar aquella idea que funcione mejor para el problema bajo consideración en lugar de adherirse en forma rígida a la filosofía de una metaheurística particular.

■ REFERENCIAS SELECCIONADAS

1. Coello, C., D. A. Van Veldhuizen y G. B. Lamont: *Evolutionary Algorithms for Solving Multi-Objective Problems*, Kluwer Academic Publishers (Springer en la actualidad), Boston, 2002.
2. Gen, M. y R. Cheng: *Genetic Algorithms and Engineering Optimization*, Wiley, Nueva York, 2000.
3. Glover, F.: “Tabu Search: A Tutorial”, en *Interfaces*, **20**(4): 74-94, julio-agosto de 1990.
4. Glover, F. y G. Kochenberger (eds.): *Handbook of Metaheuristics*, Kluwer Academic Publishers (Springer en la actualidad), Boston, MA, 2003. (Esta referencia proporciona una cobertura actualizada de todas las metaheurísticas que se consideraron en este capítulo, así como otras metaheurísticas.)
5. Glover, F. y M. Laguna: *Tabu Search*, Kluwer Academic Publishers (Springer en la actualidad), Boston, MA, 1997.

6. Gutin, G. y A. Punnen (eds.): *The Traveling Salesman Problem and Its Variations*, Kluwer Academic Publishers (Springer en la actualidad), Boston, MA, 2002.
7. Haupt, R. L. y S. E. Haupt: *Practical Genetic Algorithms*, Wiley, Nueva York, 1998.
8. Jones, D. F., S. K. Mirrazavi y M. Tamiz: "Multiobjective Metaheuristics: An Overview of the Current State of the Art", en *European Journal of Operational Research*, **137**: 1-9, 2002.
9. Laguna M. y R. Marti: *Scatter Search: Methodology and Implementations* en C, Kluwer Academic Publishers (Springer en la actualidad), Boston, 2003.
10. Michalewicz, Z. y D. B. Fogel: *How To Solve It: Modern Heuristics*, Springer, Berlín, 2002.
11. Mitchell, M.: *An Introduction to Genetic Algorithms*, MIT Press, Cambridge, MA, 1998.
12. Molina, J., M. Laguna, R. Marti y R. Caballero: "SSPMO: A Scatter Tabu Search Procedure for Non-Linear Multiobjective Optimization", en *INFORMS Journal on Computing*, **19**(1): 91-100, Winter, 2007.
13. Reeves, C. R.: "Genetic Algorithms for the Operations Researcher", en *INFORMS Journal on Computing*, **9**: 231-250, 1997. (También vea en pp. 251-265 comentarios sobre este artículo.)
14. Sarker, R., M. Mohammadian y X. Yao (eds.): *Evolutionary Optimization*, Kluwer Academic Publishers (Springer en la actualidad), Boston, MA, 2002.

■ AYUDAS DE APRENDIZAJE PARA ESTE CAPÍTULO EN EL SITIO DE INTERNET DE ESTE LIBRO (www.mhhe.com/hillier)

Ejemplos resueltos:

Ejemplos para el capítulo 13

Rutinas automáticas en el IOR Tutorial:

Algoritmo de búsqueda tabú para problemas del agente viajero (Tabu Search Algorithm for Traveling Salesman Problems)

Algoritmo de templado simulado para problemas del agente viajero (Simulated Annealing Algorithm for Traveling Salesman Problems)

Algoritmo de templado simulado para problemas de programación no lineal (Simulated Annealing Algorithm for Nonlinear Programming Problems)

Algoritmo genético para problemas de programación no lineal entera (Genetic Algorithm for Integer Nonlinear Programming Problems)

Algoritmo genético para problemas del agente viajero (Genetic Algorithm for Traveling Salesman Problems)

Glosario del capítulo 13

Vea en el apéndice 1 la documentación del software.

■ PROBLEMAS

La letra A a la izquierda de algunos de los problemas (o sus incisos) tiene el siguiente significado.

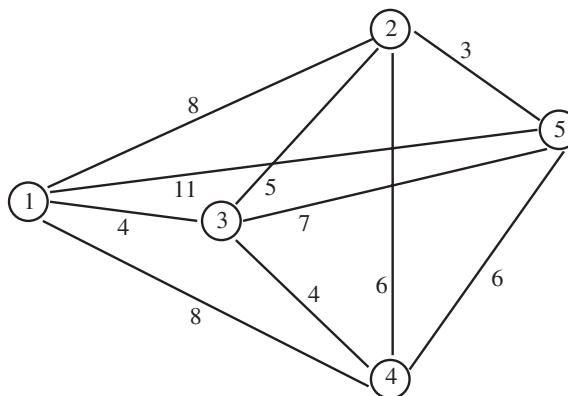
A: Se puede utilizar la rutina automática del IOR Tutorial. La impresión registra su trabajo en cada iteración.

Un asterisco después del número del problema indica que al final del libro se da al menos una respuesta parcial.

Instrucciones para obtener números aleatorios

Para cada problema o sus incisos donde se necesiten números aleatorios, obténgalos de los dígitos aleatorios consecutivos de la tabla 20.3 de la sección 20.3 de la manera siguiente. Inicie en el renglón superior de la tabla y forme números aleatorios de cinco dígitos colocando un punto decimal enfrente de cada grupo de cinco dígitos aleatorios (0.09656, 0.96657, etc.) en el orden en que se requiera los números aleatorios. Siempre reinicie desde el renglón superior para cada nuevo problema o inciso.

13.1-1. Considere el problema del agente viajero que se muestra a continuación, donde la ciudad 1 es la ciudad de residencia.

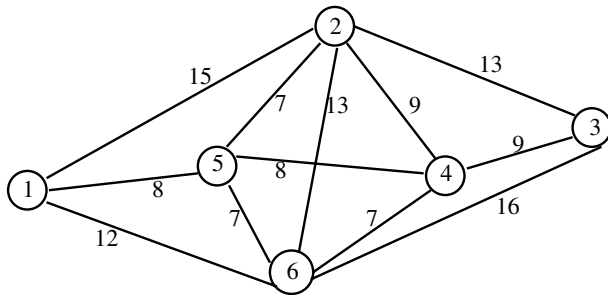


- Enumere todos los viajes posibles; excluya únicamente aquellos que sólo inviertan el orden de los viajes de la lista que se presentó con anterioridad. Calcule la distancia de cada uno de estos viajes y con esto identifique el viaje óptimo.
- Comience con 1-2-3-4-5-1 como la solución de prueba inicial para aplicar el algoritmo de subviaje inverso a este problema.
- Aplique el algoritmo de subviaje inverso a este problema; para ello comience con 1-2-4-3-5-1 como la solución de prueba inicial.
- Aplique el algoritmo de subviaje inverso a este problema; para ello comience con 1-4-2-3-5-1 como la solución de prueba inicial.

13.1-2. Reconsidere el ejemplo del problema del agente viajero que se muestra en la figura 13.4.

- Cuando se aplicó el algoritmo de subviaje inverso a este problema en la sección 13.1, la primera iteración generó un empate de cuál de los dos subviajes inversos (al invertir 3-4 o 4-5) proporcionaba la mayor reducción de la distancia del viaje; el empate se rompió de manera arbitraria a favor de la primera inversión. Determine qué hubiera pasado si se hubiese elegido la segunda de estas inversiones (al invertir 4-5).
- Aplique el algoritmo de subviaje inverso a este problema; para ello comience con 1-2-4-5-6-7-3-1 como la solución de prueba inicial.

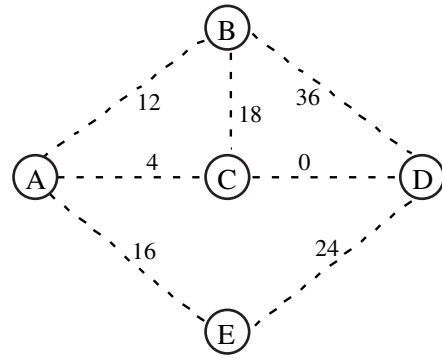
13.1-3. Considere el problema del agente viajero que se muestra a continuación, donde la ciudad 1 es la ciudad de residencia.



- Enumere todos los viajes posibles; excluya aquellos que sólo inviertan el orden de los viajes de la lista que se presentó con anterioridad. Calcule la distancia de cada uno de estos viajes y con esto identifique el viaje óptimo.
- Comience con 1-2-3-4-5-6-1 como la solución de prueba inicial para aplicar el algoritmo de subviaje inverso a este problema.
- Aplique el algoritmo de subviaje inverso a este problema; para ello comience con 1-2-5-4-3-6-1 como la solución de prueba inicial.

13.2-1. Lea el artículo de referencia que describe el estudio de IO que se resume en el recuadro de aplicación que se presentó en la sección 13.2. Describa en forma breve la manera en que la búsqueda tabú se aplicó a dicho estudio. Después, elabore una lista de los beneficios financieros y de otro tipo que arrojó este estudio.

13.2.2.* Considere el problema de árbol de expansión mínima que se presenta a continuación, donde las líneas punteadas representan las ligaduras potenciales que se podrían insertar en la red, mientras que el número en seguida de cada línea punteada es el costo asociado con la inserción de esa ligadura particular.



Este problema también tiene las siguientes dos restricciones:

Restricción 1: Sólo se puede incluir una de las ligaduras AB, BC o AE.

Restricción 2: La ligadura AB se puede incluir sólo si también está incluida la ligadura BD.

Comience con la solución de prueba inicial donde las ligaduras insertadas son AB, AC, AE y CD; después, aplique a este problema el algoritmo de búsqueda tabú básico que se presentó en la sección 13.2.

13.2-3. Reconsidere el ejemplo del problema de árbol de expansión mínima restringido que se presentó en la sección 13.2 (vea la figura 13.7a) para obtener los datos antes de introducir las restricciones). Comience con una solución de prueba inicial diferente, a saber, la que incorpora las ligaduras AB, AD, BE y CD, y aplique de nuevo el algoritmo de búsqueda tabú básico a este problema.

13.2-4. Reconsidere el ejemplo del problema de árbol de expansión mínima no restringido que se dio en la sección 9.4. Suponga que se agregan las siguientes restricciones al problema.

Restricción 1: Es obligatorio incluir la ligadura AD o la ligadura ET.

Restricción 2: Sólo se puede incluir una de las siguientes ligaduras: AO, BC o DE.

Comience con la solución óptima del problema no restringido que se presentó al final de la sección 9.4 como la solución de prueba inicial; después, aplique el algoritmo de búsqueda tabú básico a este problema.

13.2-5. Reconsidere el problema del agente viajero que se presentó en el problema 13.1-1. Aplique de forma manual el algoritmo de búsqueda tabú básico a este problema; para ello, considere 1-5-3-2-4-1 como la solución de prueba inicial.

A 13.2-6. Considere el problema del agente viajero con ocho ciudades cuyas ligaduras tienen las distancias asociadas que se muestran en la tabla siguiente (donde un guión indica la ausencia de una ligadura).

Ciudad	2	3	4	5	6	7	8
1	14	15	—	—	—	—	17
2		13	14	20	—	—	21
3			11	21	17	9	9
4				11	10	8	20
5					15	18	—
6						9	—
7							13

La ciudad 1 es la ciudad de residencia. Comience con cada una de las soluciones de prueba iniciales que se presentan en seguida para aplicar el algoritmo de búsqueda tabú básico del IOR Tutorial a este problema. En cada caso, cuente el número de veces que el algoritmo hace un movimiento sin mejora. También identifique los movimientos tabú que se realizan de cualquier modo porque generan la mejor solución de prueba encontrada hasta ese momento.

- Use 1-2-3-4-5-6-7-8-1 como la solución de prueba inicial.
- Use 1-2-5-6-7-4-8-3-1 como la solución de prueba inicial.
- Use 1-3-2-5-6-4-7-8-1 como la solución de prueba inicial.

A **13.2-7.** Considere el problema del agente viajero con 10 ciudades cuyas ligaduras tienen las distancias asociadas que se muestran en la tabla siguiente:

Ciudad	2	3	4	5	6	7	8	9	10
1	13	25	15	21	9	19	18	8	15
2		26	21	29	21	31	23	16	10
3			11	18	23	28	44	34	35
4				10	13	19	34	24	29
5					12	11	37	27	36
6						10	25	14	25
7							32	23	35
8								10	16
9									14

La ciudad 1 es la ciudad de residencia. Comience con cada una de las soluciones de prueba iniciales que se presentan a continuación para aplicar el algoritmo de búsqueda tabú básico del IOR Tutorial a este problema. En cada caso, cuente el número de veces que el algoritmo hace un movimiento sin mejora. También identifique los movimientos tabú que se realizan de cualquier modo porque generan la mejor solución de prueba que se encontró hasta ese momento.

- Use 1-2-3-4-5-6-7-8-9-10-1 como la solución de prueba inicial.
- Use 1-3-4-5-7-6-9-8-10-2-1 como la solución de prueba inicial.
- Use 1-9-8-10-2-4-3-6-7-5-1 como la solución de prueba inicial.

13.3-1. Durante la aplicación de un algoritmo de templado simulado a cierto problema, se ha llegado a una iteración donde el valor actual de T es $T = 2$ y el valor de la función objetivo de la solución de prueba actual es 30. Esta solución de prueba tiene cuatro vecinos inmediatos cuyos valores de la función objetivo son 29, 34, 31 y 24. Para cada uno de estos vecinos inmediatos, se desea determinar la probabilidad de que la regla de la selección del movimiento lo acepte como el candidato actual a ser la próxima solución de prueba, cuando se selecciona de manera aleatoria.

- Determine esta probabilidad de cada uno de los vecinos inmediatos cuando el objetivo es *maximizar* la función objetivo.
- Determine esta probabilidad de cada uno de los vecinos inmediatos cuando el objetivo es *minimizar* la función objetivo.

A **13.3-2.** Debido a que un algoritmo de templado simulado utiliza números aleatorios, proporciona resultados un poco diferentes cada vez que se corre. En la tabla 13.5 se muestra una aplicación del algoritmo de templado simulado básico del IOR Tutorial al ejemplo del problema del agente viajero que se presentó en la figura 13.4. Comience con la misma solución de prueba inicial (1-2-3-4-5-6-7-1) y use el IOR Tutorial para aplicar este algoritmo al mismo ejemplo cinco veces más. ¿Cuántas veces se encontró de nuevo la solución óptima (1-3-5-7-6-4-2-1 o de manera equivalente, 1-2-4-6-7-5-3-1)?

13.3-3. Reconsidere el problema del agente viajero que se presentó en el problema 13.1-1. Use 1-4-2-3-5-1 como la solución de prueba inicial; además, siga las instrucciones siguientes para aplicar el algoritmo de templado simulado básico que se presentó en la sección 13.3 a este problema.

- Realice la primera iteración a mano. Siga las instrucciones dadas al inicio de la sección de problemas para obtener los números aleatorios necesarios. Muestre su trabajo, en el cual debe incluir el uso de los números aleatorios.
- Use el IOR Tutorial para aplicar este algoritmo. Observe el progreso del algoritmo y registre en cada iteración cuántos candidatos a ser la próxima solución de prueba se rechazan antes de que uno sea aceptado.

A **13.3-4.** Siga las instrucciones del problema 13.3-3 del problema del agente viajero descrito en el problema 13.2-6, use 1-2-3-4-5-6-7-8-1 como la solución de prueba inicial.

A **13.3-5.** Siga las instrucciones del problema 13.3-3 para resolver el problema del agente viajero descrito en el problema 13.2-7; utilice 1-9-8-10-2-4-3-6-7-5-1 como la solución de prueba inicial.

A **13.3-6.** Debido a que un algoritmo de templado simulado utiliza números aleatorios, proporciona resultados un poco diferentes cada vez que se corre. En la tabla 13.6 se muestra una aplicación del algoritmo de templado simulado básico del IOR Tutorial al ejemplo de programación no lineal que se presentó en la sección 13.1. Comience con la misma solución de prueba inicial ($x = 15.5$) y use el IOR Tutorial para aplicar este algoritmo al mismo ejemplo cinco veces más. ¿Cuál es la mejor solución que encontró en estas cinco aplicaciones? ¿Es más cercana a la solución óptima [$x = 20$ con $f(x) = 4\,400\,000$] que la mejor solución que se muestra en la tabla 13.6?

13.3-7 Considere el siguiente problema de programación no convexa.

$$\text{Maximizar} \quad f(x) = x^3 - 60x^2 + 900x + 100,$$

sujeta a

$$0 \leq x \leq 31.$$

- Use la primera y segunda derivadas de $f(x)$ para determinar los puntos críticos (junto con los puntos extremos de la región factible) donde x es un máximo o un mínimo local.
- Bosqueje en forma manual la gráfica de $f(x)$ a lo largo de la región factible.
- Use $x = 15.5$ como la solución de prueba inicial y realice en forma manual la primera iteración del algoritmo de templado simulado que se presentó en la sección 13.3. Siga las instrucciones dadas al inicio de la sección de problemas para obtener los números aleatorios necesarios. Muestre su trabajo, incluyendo el uso de los números aleatorios.
- Use el IOR Tutorial para aplicar este algoritmo con una solución de prueba inicial de $x = 15.5$. Observe el progreso del algoritmo y registre para cada iteración cuántos candidatos a ser la próxima solución de prueba se rechazan antes de que uno sea aceptado. También cuente el número de iteraciones en las que se acepta un movimiento sin mejora.

13.3-8. Considere el ejemplo de un problema de programación no convexa que se presentó en la sección 12.10 y que se muestra en la figura 12.18.

a) Use $x = 2.5$ como la solución de prueba inicial y realice en forma manual la primera iteración del algoritmo de templado simulado que se presentó en la sección 13.3. Siga las instrucciones dadas al inicio de la sección de problemas para obtener los números aleatorios necesarios. Muestre su trabajo, que debe incluir el uso de los números aleatorios.

A b) Use el IOR Tutorial para aplicar este algoritmo con una solución de prueba inicial de $x = 2.5$. Observe el progreso del algoritmo y registre en cada iteración cuántos candidatos a ser la próxima solución de prueba se rechazan antes de que uno sea aceptado. También cuente el número de iteraciones en las que se acepta un movimiento sin mejora.

A 13.3-9. Siga las instrucciones del problema 13.3-8 para el siguiente problema de programación no convexa, con una solución de prueba inicial de $x = 25$.

$$\text{Maximizar } f(x) = x^6 - 140x^5 + 7\,000x^4 - 160\,000x^3 + 1\,600\,000x^2 - 5\,000\,000x,$$

sueto a

$$0 \leq x \leq 50.$$

A 13.3-10. Siga las instrucciones del problema 13.3-8 para resolver el siguiente problema de programación no convexa, con una solución de prueba inicial de $(x_1, x_2) = (18, 25)$.

$$\text{Maximizar } f(x_1, x_2) = x_1^5 - 81x_1^4 + 2\,330x_1^3 - 28\,750x_1^2 + 150\,000x_1 + 0.5x_2^5 - 65x_2^4 + 2\,950x_2^3 - 53\,500x_2^2 + 305\,000x_2,$$

sueto a

$$x_1 + 2x_2 \leq 110$$

$$3x_1 + x_2 \leq 120$$

y

$$0 \leq x_1 \leq 36, \quad 0 \leq x_2 \leq 50.$$

13.4-1. Para cada una de las siguientes parejas de padres genere sus dos hijos mediante la aplicación del algoritmo genético básico que se presentó en la sección 13.4 para un problema de programación no lineal entera que involucra una sola variable x , la cual está restringida a valores enteros en el intervalo $0 \leq x \leq 63$. (Siga las instrucciones dadas al principio de la sección de problemas para obtener los números aleatorios necesarios y después muestre el uso que les dio a estos números aleatorios.)

a) Los padres son 010011 y 100101.

b) Los padres son 000010 y 001101.

c) Los padres son 100000 y 101000.

13.4.2.* Considere un problema del agente viajero con ocho ciudades (ciudades 1, 2, ..., 8) donde la ciudad 1 es la ciudad de residencia y existen ligaduras entre cada par de ciudades. En el caso de cada una de las siguientes parejas de padres genere sus dos hijos mediante la aplicación del algoritmo genético básico que se presentó en la sección 13.4. (Siga las instrucciones dadas al principio de la sección de problemas para obtener los números aleatorios necesarios y después muestre el uso que dio a estos números aleatorios.)

a) Los padres son 1-2-3-4-7-6-5-8-1 y 1-5-3-6-7-8-2-4-1.

b) Los padres son 1-6-4-7-3-8-2-5-1 y 1-2-5-3-6-8-4-7-1.

c) Los padres son 1-5-7-4-6-2-3-8-1 y 1-3-7-2-5-6-8-4-1

A 13.4-3. En la tabla 13.7 se muestra la aplicación del algoritmo genético básico que se describió en la sección 13.4 a un ejemplo de programación no lineal entera durante el paso inicial y la primera iteración.

a) Use el IOR Tutorial para aplicar el mismo algoritmo al mismo ejemplo, hasta la terminación del algoritmo y a partir de otra población inicial que seleccione en forma aleatoria. ¿Esta aplicación obtiene de nuevo la solución óptima ($x = 20$), como se encontró durante la primera iteración en la tabla 13.7?

b) Debido a que un algoritmo genético utiliza números aleatorios, proporciona resultados un poco diferentes cada vez que se corre. Use el IOR Tutorial para aplicar el algoritmo genético básico descrito en la sección 13.4 a este mismo ejemplo cinco veces más. ¿Cuántas veces encuentra de nuevo la solución óptima ($x = 20$)?

13.4-4. Reconsidere el problema de programación no convexa que se presentó en el problema 13.3-7. Suponga ahora que la variable x está restringida a valores enteros.

a) Ejecute a mano el paso inicial y la primera iteración del algoritmo genético básico que se presentó en la sección 13.4. Siga las instrucciones que se dieron al principio de la sección de problemas para obtener los números aleatorios necesarios. Muestre su trabajo, el cual debe incluir el uso de los números aleatorios.

A b) Use el IOR Tutorial para aplicar este algoritmo. Observe el progreso del algoritmo y registre el número de veces que una pareja de padres produce un hijo cuya aptitud es mejor que la de ambos padres. También cuente el número de iteraciones donde la mejor solución que haya encontrado es mejor que cualquiera de las previas.

A 13.4-5. Siga las instrucciones del problema 13.4-4 para resolver el problema de programación no convexa que se describió en el problema 13.3-9, cuando la variable x está restringida a valores enteros.

A 13.4-6. Siga las instrucciones del problema 13.4-4 para resolver el problema de programación no convexa que se describió en el problema 13.3-10, cuando las dos variables x_1 y x_2 están restringidas a valores enteros.

A 13.4-7. En la tabla 13.9 se muestra la aplicación del algoritmo genético básico que se describió en la sección 13.4 al ejemplo del problema del agente viajero que se muestra en la figura 13.4 durante el paso inicial y la primera iteración del algoritmo.

a) Use el IOR Tutorial para aplicar el mismo algoritmo al mismo ejemplo, hasta la terminación del algoritmo y a partir de otra población inicial que seleccione en forma aleatoria. ¿Esta aplicación obtiene la solución óptima (1-3-5-7-6-4-2-1 o, la equivalente, 1-2-4-6-7-5-3-1)?

b) Debido a que un algoritmo genético utiliza números aleatorios, proporciona resultados un poco diferentes cada vez que se corre. Use el IOR Tutorial para aplicar el algoritmo genético básico que se describió en la sección 13.4 a este mismo ejemplo cinco veces más. ¿Cuántas veces encuentra de nuevo la solución óptima?

13.4-8. Reconsidere el problema del agente viajero que se presentó en el problema 13.1-1.

a) Ejecute a mano el paso inicial y la primera iteración del algoritmo genético básico que se presentó en la sección 13.4. Siga las instrucciones que se dieron al principio de la sección de problemas para obtener los números aleatorios necesarios. Muestre su trabajo, incluyendo el uso de los números aleatorios.

A **b)** Use el IOR Tutorial para aplicar este algoritmo. Observe el progreso del algoritmo y registre el número de veces que una pareja de padres produce un hijo cuyo viaje tiene una distancia menor que la de los dos padres. También cuente el número de iteraciones donde la mejor solución que haya encontrado tiene una distancia menor que cualquiera de las previas.

A **13.4.9.** Siga las instrucciones del problema 13.4-8 para solucionar el problema del agente viajero que se describe en el problema 13.2-6.

A **13.4.10.** Siga las instrucciones del problema 13.4-8 para resolver el problema del agente viajero que se describe en el problema 13.2-7.

A **13.5-1.** Use el IOR Tutorial para aplicar el algoritmo básico de las tres metaheurísticas que se presentó en este capítulo al problema del agente viajero que se describe en el problema 13.2-6. (Use 1-2-3-4-5-6-7-8-1 como la solución de prueba inicial para los algoritmos de búsqueda tabú y templado simulado.) ¿Cuál metaheurística proporciona la mejor solución para este problema particular?

A **13.5-2.** Use el IOR Tutorial para aplicar el algoritmo básico de las tres metaheurísticas que se presentó en este capítulo al problema del agente viajero que se describió en el problema 13.2-7. (Use 1-2-3-4-5-6-7-8-9-10-1 como la solución de prueba inicial para los algoritmos de búsqueda tabú y templado simulado.) ¿Cuál metaheurística proporciona la mejor solución para este problema particular?

14

CAPÍTULO

Teoría de juegos

La vida está llena de conflictos y competencia. Los numerosos ejemplos que involucran adversarios en conflicto incluyen juegos de mesa, combates militares, campañas políticas, competencias deportivas, campañas de publicidad y comercialización en las competencias de empresas, entre otros. Una característica básica en muchas de estas situaciones es que el resultado final depende, en primer lugar, de la combinación de estrategias seleccionadas por los adversarios. La teoría de juegos es una teoría matemática que estudia las características generales de situaciones competitivas de manera formal y abstracta. Además, otorga una importancia especial a los procesos de toma de decisiones de los adversarios.

Debido a que los escenarios de competencia son algo muy común en la actualidad, la teoría de juegos tiene aplicaciones en una gran variedad de áreas, dentro de las cuales se incluyen los negocios y la economía. Por ejemplo, la referencia seleccionada 3 presenta varias aplicaciones de negocios de la teoría de juegos y la referencia seleccionada 1 se enfoca en sus aplicaciones a la economía. El premio Nobel en Economía de 1994 le fue otorgado a John F. Nash, Jr. (cuya biografía se relata en la película *A Beautiful Mind*), John C. Harsanyi y Reinhard Selton por su análisis de equilibrio en la teoría de juegos no cooperativos. Tiempo después, en 2005, Robert J. Aumann y Thomas C. Schelling ganaron el premio Nobel en la misma disciplina por la mejora que aportaron a la comprensión del conflicto y la cooperación a través del análisis de la teoría de juegos.

Como se analiza de manera breve en la sección 14.6, la investigación sobre teoría de juegos continúa con el sondeo de las situaciones competitivas de tipo complicado. No obstante, este capítulo se aboca al caso más sencillo conocido como **juegos de dos personas y suma cero**. Como su nombre lo indica, en estos juegos participan sólo dos adversarios o *jugadores* (que pueden ser ejércitos, equipos, empresas, etc.). Son llamados juegos de *suma cero* porque un jugador gana lo que el otro pierde, de manera que la suma de sus ganancias netas es cero.

Después de introducir el modelo básico de los juegos de dos personas y suma cero en la sección 14.1, en las cuatro secciones siguientes se describen e ilustran distintos enfoques para resolver este tipo de juegos. El capítulo concluye con la mención de otro tipo de situaciones competitivas que se estudian en otras ramas de la teoría de juegos.

14.1 FORMULACIÓN DE JUEGOS DE DOS PERSONAS Y SUMA CERO

Para ilustrar las características básicas de un modelo de juegos de dos personas de suma cero, considere el juego llamado *pares y nones*. Éste consiste nada más en que los dos jugadores muestran al mismo tiempo uno o dos dedos. Si el número total de dedos mostrados por ambos jugadores es par, el jugador que apuesta a pares (por ejemplo, el jugador 1) gana la apuesta (digamos 1 dólar) al jugador que elige nones (jugador 2). Si el número de dedos es impar, el jugador 1 paga 1 dólar al jugador 2. Entonces, cada jugador tiene dos *estrategias*: mostrar uno o dos dedos. El pago en dólares que resulta para el jugador 1 se muestra en una *matriz de pagos* en la tabla 14.1.

■ **TABLA 14.1** Matriz de pagos del juego de pares y nones

Estrategia	Jugador 2	
	1	2
Jugador 1		
1	1	-1
2	-1	1

En general, un juego de dos personas se caracteriza por

1. Las estrategias del jugador 1.
2. Las estrategias del jugador 2.
3. La matriz de pagos.

Antes de iniciar el juego, cada jugador conoce las estrategias de que dispone, las que tiene su oponente y la matriz de pagos. Una jugada real consiste en que los dos jugadores elijan al mismo tiempo una estrategia sin saber cuál es la elección de su oponente.

Una estrategia puede constituir una acción sencilla, como mostrar un número par o non de dedos en el juego de pares y nones. Por otro lado, en juegos más complicados que llevan en sí una serie de movimientos, una **estrategia** es una regla predeterminada que especifica por completo cómo se intenta responder a cada circunstancia posible en cada etapa del juego. Por ejemplo, una estrategia de un jugador de ajedrez indica cómo hacer el siguiente movimiento ante *todas* las posiciones posibles en el tablero, por lo que el número total de estrategias posibles sería astronómico. Las aplicaciones de la teoría de juegos involucran situaciones competitivas mucho menos complicadas que el ajedrez, pero las estrategias que se manejan pueden llegar a ser bastante complejas.

Por lo general, la **matriz de pagos** muestra la ganancia (positiva o negativa) del jugador 1, que resultaría con cada combinación de estrategias de los dos jugadores. Se presenta sólo la matriz del jugador 1, puesto que la del jugador 2 es el negativo de ésta, debido a la naturaleza de suma cero del juego.

Los elementos de la matriz de pagos pueden expresar cualquier tipo de unidades, como dólares, siempre que representen con exactitud la *utilidad* del jugador 1 ante el resultado correspondiente. Debe hacerse hincapié en que la utilidad no necesariamente es proporcional a la cantidad de dinero (o cualquier otro bien) cuando se manejan cantidades grandes. Por ejemplo, 2 millones de dólares (después de impuestos) pueden tener un valor mucho menor que “el doble” del valor que representa 1 millón de dólares para una persona pobre. En otras palabras, si a una persona se le da a elegir entre: 1) recibir, con 50% de posibilidades, 2 millones o nada y 2) recibir 1 millón con seguridad, una persona pobre tal vez preferiría la última oferta. En otras palabras, el resultado que corresponde a un elemento con valor 2 en una matriz de pagos debe “valer el doble” para el jugador 1 que el resultado correspondiente a un elemento de 1. Así, dada la elección, debe serle indiferente 50% de posibilidades de recibir el primer resultado (en lugar de nada) y recibir en definitiva el último resultado.¹

Un objetivo primordial de la teoría de juegos es desarrollar criterios racionales para seleccionar una estrategia, los cuales implican dos supuestos importantes:

1. *Ambos jugadores son racionales.*
2. *Ambos jugadores eligen sus estrategias sólo para promover su propio bienestar (sin compasión para el oponente).*

La teoría de juegos se contrapone al *análisis de decisión* (vea el capítulo 15), en donde se hace el supuesto de que el tomador de decisiones está jugando un juego contra un oponente pasivo —la naturaleza— que elige sus estrategias de alguna manera aleatoria.

¹ Vea en la sección 15.6 una presentación más completa del concepto de utilidad.

En este capítulo se desarrollará el criterio estándar de la teoría de juegos para elegir las estrategias mediante ejemplos ilustrativos. En particular, al final de la siguiente sección se describe la forma en que la teoría de juegos dice cómo debe jugarse el juego de pares y nones. (Los problemas 14.3-1, 14.4-1 y 14.5-1 lo invitan también a aplicar las técnicas desarrolladas en este capítulo para encontrar la forma óptima de acometer este juego.) Además, la sección siguiente presenta un ejemplo prototipo que ilustra la formulación de un juego y su solución en algunas situaciones sencillas. Después, en la sección 14.3, se desarrollará una variación más complicada de este juego para obtener un criterio más general. En las secciones 14.4 y 14.5 se describe un procedimiento gráfico y una formulación de programación lineal para juegos de este tipo.

■ 14.2 SOLUCIÓN DE JUEGOS SENCILLOS: EJEMPLO PROTOTIPO

Dos políticos contenden entre sí por un lugar en el Senado de Estados Unidos. En este momento elaboran sus planes de campaña para los dos últimos días antes de las elecciones; se espera que dichos días sean cruciales debido a que están muy próximos al día de la votación. Por esta circunstancia, ambos quieren emplearlos para hacer campaña en dos ciudades importantes: Bigtown y Megalópolis. Para evitar pérdidas de tiempo, planean viajar en la noche y pasar un día completo en cada ciudad o dos días en sólo una de ellas. Como deben hacer los arreglos necesarios por adelantado, ninguno de los dos conocerá lo que su oponente tiene planeado hacer hasta después de concretar sus propios planes. Cada político tiene un jefe de campaña en cada ciudad para asesorarlo sobre el efecto que tendrán (en términos de votos ganados o perdidos) las combinaciones posibles de los días dedicados a cada ciudad por ellos o por sus oponentes. Por tanto, quieren emplear esta información para elegir su mejor estrategia para estos dos días.

Formulación como un juego de dos personas y suma cero

Para formular este problema como un juego de dos personas y suma cero se deben identificar los *dos jugadores* (obviamente, los dos políticos), las *estrategias* de cada uno de ellos y la *matriz de pagos*.

Según la forma en que se estableció el problema, cada jugador tiene tres estrategias:

Estrategia 1 = pasar un día en cada ciudad.

Estrategia 2 = pasar los dos días en Bigtown.

Estrategia 3 = pasar los dos días en Megalópolis.

Por el contrario, las estrategias serían más complicadas en una situación diferente en la que cada político supiera en dónde pasará su oponente el primer día antes de concluir sus propios planes para el segundo día. En ese caso, una estrategia normal sería: pasar el primer día en Bigtown; si el oponente también pasa el día en Bigtown, entonces quedarse el segundo día en esa ciudad; sin embargo, si el oponente pasa el primer día en Megalópolis, entonces sería necesario pasar el segundo día en ese lugar. Habría ocho estrategias de este tipo, una para cada combinación de las dos posibilidades para el primer día, las dos para el primer día del oponente y las dos alternativas para el segundo día.

Cada elemento de la matriz de pagos del jugador 1 representa la *utilidad* para ese jugador (o la utilidad negativa para el jugador 2) de los resultados que se obtienen cuando los dos jugadores emplean las estrategias correspondientes. Desde el punto de vista de los políticos, el objetivo es *ganar votos* y cada voto adicional (antes de conocer el resultado de las elecciones) tiene el mismo valor para él. Entonces, los elementos apropiados de la matriz de pagos se darán en términos del *total neto de votos ganados* a su oponente (esto es, la suma de la cantidad neta de cambios de votos en las dos ciudades) que resulte de estos dos días de campaña. Con unidades de 1 000 votos, esta formulación se resume en la tabla 14.2. La teoría de juegos supone que ambos jugadores usan la misma formulación (incluso los mismos pagos para el jugador 1) para elegir sus estrategias.

Sin embargo, también debe hacerse notar que esta matriz de pagos *no* sería apropiada si se contara con información adicional. En particular, suponga que se conoce con exactitud cuáles son los planes de votación de la población dos días antes de las elecciones, de manera que cada político sabe la cantidad neta de votos (positiva o negativa) que necesita cambiar a su favor durante los dos últimos días de campaña para ganar. Entonces, lo único significativo de los datos que se pro-

■ **TABLA 14.2** Formulación de la matriz de pagos del problema de la campaña política

Estrategia	Cantidad neta en unidades de votos ganados por el político 1 (cada unidad equivale a 1 000 votos)		
	Político 2		
	1	2	3
Político 1	1	2	3

porcionan en la tabla 14.2 sería que indican cuál es el político que ganaría las elecciones con cada combinación de estrategias. Como la meta final es ganar la elección y como el tamaño de la mayoría no tiene consecuencias, los elementos de utilidad de la matriz deben ser constantes positivas (digamos +1) cuando el político 1 gana y -1 cuando pierde. Aun cuando sólo se pudiera determinar una *probabilidad* de ganar para cada combinación de estrategias, los elementos apropiados de la matriz serían la probabilidad de ganar menos la probabilidad de perder, pues de esta forma representarían las utilidades *esperadas*. Sin embargo, casi nunca se dispone de datos con suficiente exactitud como para hacer ese tipo de determinaciones, por lo que este ejemplo utiliza los miles de votos netos totales ganados por el político 1 como elementos de la matriz de pagos.

Cuando se utiliza la forma dada en la tabla 14.2 se obtienen tres conjuntos de datos alternativos para la matriz de pagos, a fin de ilustrar cómo se resuelven tres tipos distintos de juegos.

Variación 1 del ejemplo

Si la tabla 14.3 representa la matriz de pagos del jugador 1 (político 1), ¿qué estrategia debe elegir cada uno?

Esta situación es bastante especial; en ella se puede obtener la respuesta con sólo aplicar el concepto de **estrategia dominada** para eliminar una serie de estrategias inferiores hasta que quede sólo una que se pueda elegir.

Una estrategia es **dominada** por una segunda estrategia si esta última es *siempre al menos tan buena* (y algunas veces mejor) como la primera, sin que importe lo que haga el oponente. Una estrategia dominada se puede eliminar de inmediato para consideraciones posteriores.

En principio, la tabla 14.3 no incluye las estrategias dominadas del jugador 2; pero para el jugador 1, la estrategia 3 está dominada por la estrategia 1, ya que tiene pagos más altos ($1 > 0$, $2 > 1$, $4 > -1$), independientemente de lo que haga el jugador 2. Al eliminar la estrategia 3 se obtiene la siguiente matriz de pagos reducida:

	1	2	3
1	1	2	4
2	1	0	5

■ **TABLA 14.3** Matriz de pagos de la variación 1 del problema de la campaña política

Estrategia	Jugador 2		
	1	2	3
Jugador 1	1	2	4
2	1	0	5
3	0	1	-1

Como se supone que ambos jugadores son racionales, también el jugador 2 puede deducir que el jugador 1 sólo dispone de estas dos estrategias. Entonces, ahora el jugador 2 *tiene* una estrategia dominada: la estrategia 3, que está dominada tanto por la estrategia 1 como por la 2 puesto que siempre tienen menores pérdidas (pagos al jugador 1) en esta matriz de pagos reducida (para la estrategia 1: $1 < 4$, $1 < 5$; para la estrategia 2: $2 < 4$, $0 < 5$). Cuando se elimina esta estrategia se obtiene

	1	2
1	1	2
2	1	0

En este punto, la estrategia 2 del jugador 1 se convierte en dominada por la estrategia 1, puesto que esta última es mejor en la columna 2 ($2 > 0$) y es igual en la columna 1 ($1 = 1$). Si se elimina esta estrategia dominada se llega a

	1	2
1	1	2

Ahora la estrategia 2 del jugador 2 está dominada por la estrategia 1 ($1 < 2$), por lo que debe eliminarse la estrategia 2.

En consecuencia, ambos jugadores deberán elegir su estrategia 1. Con esta solución, el jugador 1 recibirá un pago de 1 por parte del jugador 2 (esto es, el político 1 ganará 1 000 votos al político 2).

Si usted quiere ver **otro ejemplo** donde se muestre cómo resolver un juego utilizando el concepto de estrategias dominantes, en la sección Worked Examples del sitio en internet de este libro se proporciona uno.

En general, el pago para el jugador 1 cuando ambos jugadores operan de manera óptima recibe el nombre de **valor del juego**. Se dice que se trata de un **juego justo** cuando el juego tiene valor 0. Como este juego en particular tiene valor 1, *no* es un juego justo.

El concepto de estrategia dominada es muy útil para reducir el tamaño de la matriz de pagos en cuestión y en algunos casos raros como éste puede, de hecho, identificar la solución óptima del juego. Sin embargo, casi todos los juegos requieren otro enfoque para terminar de resolverlos, como se ilustra en las dos variaciones siguientes del ejemplo.

Variación 2 del ejemplo

Ahora suponga que los datos actuales se dan en la tabla 14.4 como la matriz de pagos del jugador 1 (político 1). Este juego no tiene estrategias dominadas por lo que no es obvio qué deben hacer los jugadores. ¿Qué línea de razonamiento dice la teoría de juegos que debe usarse en este caso?

Considere al jugador 1. Si elige la estrategia 1 puede ganar 6 o perder 3. Como el jugador 2 es racional y tratará de aplicar una estrategia que lo proteja de pagos grandes al jugador 1, parece probable que si juega la estrategia 1, el jugador 1 perderá. De manera similar, si selecciona la estrategia 3 el jugador 1 puede ganar 5, pero quizá sea más probable que su oponente racional evite esta pérdida y en su lugar logre que él pierda, lo que puede ascender a 4. Por otro lado, si el jugador 1 elige la estrategia 2, tiene garantizado que no perderá y quizá gane algo. Entonces, por proporcio-

■ **TABLA 14.4** Matriz de pagos del jugador 1 en la variación 2 del problema de la campaña política

		Jugador 2			Mínimo
Estrategia		1	2	3	
Jugador 1	1	-3	-2	6	-3
	2	2	0	2	0 ← Valor maximin
	3	5	-2	-4	-4
Máximo: 5			0	6	
			↑		
			Valor minimax		

nar la *mejor garantía* (un pago de 0), la estrategia 2 parece ser la elección “racional” del jugador 1 contra su oponente racional. (Esta línea de razonamiento supone que ambos jugadores tienen aversión a arriesgar pérdidas más grandes que las necesarias, al contrario de quienes disfrutan la apuesta por un gran pago con pocas posibilidades.)

Ahora considere al jugador 2. Puede perder tanto como 5 o 6 si sigue las estrategias 1 o 3, pero está garantizado que al menos empata con la estrategia 2. Entonces, si usa el mismo razonamiento para buscar su mejor garantía contra su oponente racional, parece que la mejor elección es la estrategia 2.

Si los dos jugadores eligen la estrategia 2, el resultado es que empatan. Así, en este caso, ningún jugador mejora con su mejor garantía, pero ambos fuerzan al oponente hacia la misma posición. Aunque cada jugador deduzca la estrategia del otro, no puede explotar esta información para mejorar su posición. Se trata de un empate.

El producto final de esta línea de razonamiento es que cada jugador debe jugar de tal manera que *minimice su pérdida máxima*, siempre que el resultado de su elección no pueda ser aprovechado por su oponente para mejorar su posición. Esto se conoce como **criterio minimax**, criterio estándar que propone la teoría de juegos para elegir una estrategia. En realidad, este criterio sostiene que se debe seleccionar la mejor estrategia, aun cuando la elección fuera anunciada al oponente antes de que éste eligiera su estrategia. En términos de la matriz de pagos, implica que el *jugador 1* debe elegir aquella estrategia cuyo *pago mínimo* sea el *mayor*, mientras que el *jugador 2* debe elegir aquella cuyo *pago máximo al jugador 1* sea el *menor*. Este criterio se muestra en la tabla 14.4, donde se identifica la estrategia 2 como la *estrategia maximin* del jugador 1, y la estrategia 2 es la *estrategia minimax* del jugador 2. El pago de 0 que resulta es el valor del juego, por lo que éste es un juego justo.

Observe que en esta matriz de pagos el mismo elemento proporciona tanto el valor mínimo como el máximo. Este interesante hecho se debe a que tal elemento, por un lado, es el mínimo del renglón y, por el otro, el máximo de la columna. Esta posición de un elemento se llama **punto silla**.

El hecho de que este juego posea un punto silla es en realidad esencial para determinar cómo debe jugarse. A causa de este punto silla, ningún jugador puede aprovechar la estrategia de su oponente para mejorar su propia posición. En particular, si el jugador 2 predice o sabe que el jugador 1 empleará la estrategia 2, y cambia su plan original de usar la estrategia 2 sólo aumentará sus pérdidas. De igual manera, el jugador 1 sólo empeoraría su posición si cambiara su plan. Así, ningún jugador tiene motivos para considerar un cambio de estrategias, para quedar con ventaja respecto de su oponente, o para evitar que su oponente tenga ventaja. Entonces, ésta es una **solución estable** (llamada también *solución de equilibrio*), y cada jugador debe, exclusivamente, emplear sus respectivas estrategias maximin y minimax.

La siguiente variación ilustrará que algunos juegos no tienen punto silla y que en ese caso se requiere de un análisis más complicado.

Variación 3 del ejemplo

La información reciente sobre la campaña da como resultado la matriz de pagos final de los dos políticos (jugadores) que se muestra en la tabla 14.5. ¿Cómo debe jugarse este juego?

■ TABLA 14.5 Matriz de pagos del jugador 1 de la variación 3 del problema de la campaña política

		Jugador 2			Mínimo
		1	2	3	
Jugador 1	1	0	−2	2	−2 ← Valor maximin
	2	5	4	−3	−3
	3	2	3	−4	−4
Máximo: 5			4	2 ↑ Valor minimax	

Suponga que ambos jugadores quieren aplicar el criterio minimax igual que en la variación 2. El jugador 1 puede garantizar que no perderá más de 2 si juega la estrategia 1. De la misma manera, el jugador 2 puede asegurar que no perderá más de 2 si elige la estrategia 3.

Sin embargo, observe que, en este caso, el valor máximo (-2) y el valor mínimo (2) no coinciden. El resultado es que *no hay punto silla*.

¿Cuáles serán las consecuencias si ambos jugadores planean usar las estrategias mencionadas? Se puede observar que el jugador 1 ganaría 2 al jugador 2, lo que molestaría a este último. Como éste es racional, puede prever el resultado, con lo que concluiría que puede actuar mejor si gana 2 en lugar de perder 2 al jugar la estrategia 2. Como el jugador 1 también es racional, prevendría este cambio y concluiría que él también puede mejorar mucho, de -2 a 4 , si cambia a la estrategia 2. Al darse cuenta de esto, el jugador 2 tomaría en cuenta regresar a la estrategia 3 para convertir la pérdida de 4 en una ganancia de 3 . Este cambio posible causaría que el jugador 1 usara de nuevo la estrategia 1, después de lo cual volvería a comenzar todo el ciclo. Por lo tanto, aunque este juego se juega sólo una vez, *cualquier* elección tentativa de una estrategia deja al jugador en posición de considerar un cambio de estrategia, ya sea para tener ventaja sobre su oponente o para evitar que el oponente tenga ventaja sobre él.

En pocas palabras, la solución que se sugirió al principio —estrategia 1 para el jugador 1 y estrategia 3 para el jugador 2— es una **solución inestable**; con esto se ve la necesidad de desarrollar una solución más satisfactoria. Pero, ¿qué clase de solución debe ser ésta?

El hecho fundamental parece ser que siempre que se puede predecir la estrategia de un jugador, el oponente puede aprovechar esta información para mejorar su posición. Por lo tanto, una característica esencial de un plan racional para jugar un juego como éste es que ningún competidor pueda predecir qué estrategia usará el otro. Así, en este caso, en lugar de aplicar algún criterio conocido para determinar una sola estrategia que se usará en forma definitiva, es necesario elegir entre las estrategias aceptables de alguna manera aleatoria. Al hacer esto, ningún jugador conoce de antemano cuál de sus propias estrategias se usará, mucho menos la de su oponente.

Se plantea la misma situación con el juego de pares y nones que se presentó en la sección 14.1. La tabla de pagos para este juego que se muestra en la tabla 14.1 no tiene un punto de silla, por lo que el juego no tiene una solución estable en relación a qué estrategia (mostrar uno o dos dedos) debe seleccionar cada jugador en cada jugada. En realidad, sería una tontería que un jugador siempre mostrara el mismo número de dedos ya que entonces el oponente podría empezar a mostrar siempre el número de dedos que ganaría cada vez. Aun si la estrategia de un jugador fuera predecible de alguna forma debido a tendencias pasadas o patrones, el oponente puede aprovechar esta información para mejorar sus posibilidades de ganar. De acuerdo con la teoría de juegos, la forma racional de jugar al juego de pares y nones consiste en hacer que la elección de la estrategia sea completamente aleatoria cada vez. Esta situación se puede lograr, por ejemplo, si se lanza al aire una moneda (sin mostrar el resultado al oponente) y después se muestra, digamos, un dedo si la moneda cayó del lado del águila y dos dedos si cayó del otro lado.

Esto sugiere, en términos muy generales, el tipo de enfoque que se requiere para juegos sin punto silla. La siguiente sección presenta este enfoque en forma más completa. Dadas estas bases, en las siguientes dos secciones se desarrollarán procedimientos para encontrar la manera óptima de jugar estos juegos. Se continuará con el uso esta variación específica del problema de la campaña política con el fin de ejemplificar las ideas a medida que éstas se desarrollen.

14.3 JUEGOS CON ESTRATEGIAS MIXTAS

Cuando un juego no tiene punto silla, la teoría de juegos aconseja a cada jugador asignar una distribución de probabilidad sobre su conjunto de estrategias. Para expresar este consejo de manera matemática, sea

$$\begin{aligned} x_i &= \text{probabilidad de que el jugador 1 use la estrategia } i \ (i = 1, 2, \dots, m), \\ y_j &= \text{probabilidad de que el jugador 2 use la estrategia } j \ (j = 1, 2, \dots, n), \end{aligned}$$

donde m y n son los números respectivos de estrategias disponibles. Entonces, el jugador 1 especificará su plan de juego al asignar valores a $x_1, x_2, \dots, x_m$. Como estos valores son probabilidades, tendrán que ser no negativos y sumar 1. De igual manera, el plan del jugador 2 se describe mediante

los valores que asigne a sus variables de decisión $y_1, y_2, \dots, y_n$. Por lo general se hace referencia a estos planes $(x_1, x_2, \dots, x_m)$ y $(y_1, y_2, \dots, y_n)$ con el nombre de **estrategias mixtas**, y las estrategias originales se llaman **estrategias puras**.

En el momento de jugar, es necesario que cada participante use una de sus estrategias puras, pero debe elegirla mediante algún dispositivo aleatorio para obtener una observación aleatoria que siga la distribución de probabilidad que especifica la estrategia mixta; esta observación indicará la estrategia pura que se debe usar.

A manera de ilustración, suponga que los jugadores 1 y 2 en la variación 3 del problema de la campaña política (vea la tabla 14.5) eligen las estrategias mixtas $(x_1, x_2, x_3) = (\frac{1}{2}, \frac{1}{2}, 0)$ y $(y_1, y_2, y_3) = (0, \frac{1}{2}, \frac{1}{2})$, respectivamente. Esta selección indica que el jugador 1 da igual oportunidad (probabilidad de $\frac{1}{2}$) a la elección de las estrategias (puras) 1 o 2, pero que descarta por completo la estrategia 3. De manera análoga, el jugador 2 elige al azar entre sus dos últimas estrategias puras. Para llevar a cabo la jugada, cada jugador podría tirar una moneda al aire para determinar cuál de sus dos estrategias aceptables usará.

Aunque no se cuenta con una medida de desempeño satisfactoria por completo para evaluar las estrategias mixtas, el *pago esperado* es una herramienta útil. Al aplicar la definición de valor esperado de la teoría de probabilidad, esta cantidad es

$$\text{Pago esperado para el jugador 1} = \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_i y_j,$$

donde p_{ij} es el pago si el jugador 1 usa la estrategia pura i y el jugador 2 usa la estrategia pura j . En el ejemplo de estrategias mixtas que se acaba de dar existen cuatro pagos posibles $(-2, 2, 4, -3)$, donde cada uno ocurre con una probabilidad de $\frac{1}{4}$, el pago esperado es $\frac{1}{4}(-2 + 2 + 4 - 3) = \frac{1}{4}$. Así, esta medida de desempeño no revela nada sobre los riesgos inherentes al juego, pero indica a qué cantidad tendería el pago promedio si el juego se efectuara muchas veces.

Con esta medida, la teoría de juegos puede extender el concepto del **criterio minimax** a juegos que no tienen punto silla y que, por tanto, necesitan estrategias mixtas. En este contexto, el criterio minimax sostiene que un jugador debe elegir la estrategia mixta que *minimice la máxima pérdida que espera* para sí mismo. De manera equivalente, si se analizan los pagos (jugador 1) en lugar de las pérdidas (jugador 2), este criterio es *maximin*, es decir, *maximizar el pago esperado mínimo* para el jugador. Por *pago esperado mínimo* se entiende el pago esperado más pequeño posible que puede resultar de cualquier estrategia mixta con la que el oponente puede contar. De esta forma, la estrategia mixta del jugador 1 que es *óptima*, según este criterio, es la que proporciona la *garantía* (el mínimo pago esperado) de que es la *mejor* (máxima). (El valor de esta mejor garantía es el *valor maximin* y se denota por $\underline{v}$.) De manera similar, la estrategia *óptima* del jugador 2 es la que proporciona la *mejor garantía*, donde *mejor* significa *mínima* y *garantía* se refiere a la *máxima pérdida que espera* poder lograr con cualquiera de las estrategias mixtas del oponente. (La mejor garantía es el *valor minimax* que se denota por $\bar{v}$.)

Recuerde que cuando sólo se usan estrategias puras, resulta que los juegos que no tienen punto silla son *inestables* (sin soluciones estables). La razón esencial es que $\underline{v} < \bar{v}$, por lo que los jugadores quieren cambiar sus estrategias para mejorar su posición. De manera parecida, en los juegos con estrategias mixtas es necesario que $\underline{v} = \bar{v}$ para que la solución óptima sea *estable*. Por fortuna, según el teorema minimax de teoría de juegos, esta condición siempre se cumple para estos juegos.

Teorema minimax: Si se permiten estrategias mixtas, el par de estrategias que es óptimo de acuerdo con el criterio minimax proporciona una *solución estable* con $\underline{v} = \bar{v} = v$ (el valor del juego), de manera que ninguno de los dos jugadores puede mejorar si cambia de manera unilateral su estrategia.

En la sección 14.5 se incluye una demostración de este teorema.

Aunque el concepto de estrategias mixtas se convierte en un proceso bastante intuitivo si es jugado *repetidas veces*, si requiere de una interpretación cuando sólo va a ser jugado *una vez*. En este caso, el uso de una estrategia mixta todavía implica elegir y usar *una* estrategia pura, elegida en forma aleatoria a partir de una distribución de probabilidad específica, y podría parecer más sensato pasar por alto este proceso de aleatorización y sólo escoger la “mejor” estrategia pura. Sin embargo, ya se ilustró en la variación 3 de la sección anterior que un jugador *no* puede permitir

que su oponente deduzca cuál será su estrategia (es decir, el procedimiento de solución bajo las reglas de teoría de juegos no debe identificar de manera *definitiva* la estrategia pura que se usará, cuando el juego es inestable). Todavía más, aun cuando el oponente pueda usar sólo su conocimiento de las tendencias del primer jugador para deducir probabilidades —de la estrategia pura elegida— que sean distintas a las de estrategia mixta óptima, entonces el oponente todavía puede aprovechar este conocimiento para reducir el pago esperado para el primer jugador. Así pues, la única manera de garantizar que se logre el pago esperado óptimo v , es elegir de manera aleatoria la estrategia pura que debe usarse, a partir de la distribución de probabilidad de la estrategia mixta óptima. (En la sección 20.4 se estudian los procedimientos estadísticos válidos para llevar a cabo esa selección aleatoria.)

A continuación se mostrará cómo cada jugador encuentra su estrategia mixta óptima. Se dispone de varios métodos. Uno es un procedimiento gráfico que se puede usar siempre que uno de los jugadores tenga sólo dos estrategias puras (no dominadas); este enfoque se describe en la siguiente sección. Cuando se trata de juegos más grandes, el método que más se emplea consiste en transformar el problema en uno de programación lineal que se puede resolver por el método símplex en una computadora; en la sección 14.5 se analiza este enfoque.

14.4 PROCEDIMIENTO DE SOLUCIÓN GRÁFICO

Considere cualquier juego con estrategias mixtas tal que después de eliminar las estrategias dominadas, uno de los jugadores tiene sólo dos estrategias puras. Para ser específico, sea éste el jugador 1. Como sus estrategias mixtas son (x_1, x_2) y $x_2 = 1 - x_1$, nada más debe obtener el valor óptimo de x_1 . Sin embargo, resulta directo y sencillo hacer la gráfica del pago esperado como una función de x_1 , para cada una de las estrategias de su oponente. Esta gráfica se puede usar para identificar el punto que maximiza el mínimo pago esperado. También es posible identificar en ella la estrategia mixta minimax del oponente.

Para ilustrar este procedimiento, considere la variación 3 del problema de la campaña política (vea la tabla 14.5). Observe que la tercera estrategia pura del jugador 1 está dominada por la segunda, por lo que la matriz de pagos se puede reducir a la forma dada en la tabla 14.6. Entonces, para cada estrategia pura de que dispone el jugador 2, el pago esperado para el jugador 1 será:

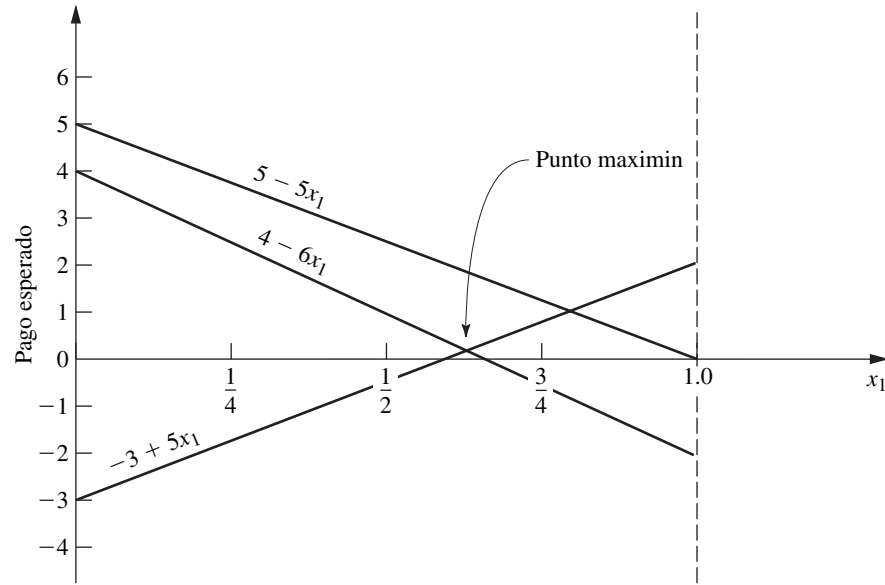
(y_1, y_2, y_3)	Pago esperado
$(1, 0, 0)$	$0x_1 + 5(1 - x_1) = 5 - 5x_1$
$(0, 1, 0)$	$-2x_1 + 4(1 - x_1) = 4 - 6x_1$
$(0, 0, 1)$	$2x_1 - 3(1 - x_1) = -3 + 5x_1$

A continuación se grafican estas rectas del pago esperado, como se muestra en la figura 14.1. Para cualquier valor dado de x_1 y de (y_1, y_2, y_3) , el pago esperado será el promedio ponderado apropiado de los puntos correspondientes a estas tres rectas. En particular,

$$\text{Pago esperado para el jugador 1} = y_1(5 - 5x_1) + y_2(4 - 6x_1) + y_3(-3 + 5x_1).$$

TABLA 14.6 Tabla de pagos reducida del jugador 1 de la variación 3 del problema de la campaña política

		Jugador 2		
		y_1	y_2	y_3
Probabilidad	Estrategia pura	1	2	3
Jugador 1	x_1	0	-2	2
	$1 - x_1$	5	4	-3



■ **FIGURA 14.1**
Procedimiento gráfico para
la solución de los juegos.

Recuerde que el jugador 2 quiere minimizar este pago esperado para el jugador 1. Dado x_1 , el jugador 2 puede minimizar este pago esperado si elige la estrategia pura que corresponde a la recta “inferior” de esa x_1 en la figura 14.1 (ya sea $-3 + 5x_1$ o $4 - 6x_1$, pero no $5 - 5x_1$). Según el criterio minimax (o maximin), el jugador 1 quiere maximizar este pago esperado mínimo. En consecuencia, debe elegir el valor de x_1 para el que la recta inferior tenga su mayor valor, es decir, el valor en el que las rectas $(-3 + 5x_1)$ y $(4 - 6x_1)$ se cruzan, de donde se obtiene un pago esperado de

$$\underline{v} = v = \max_{0 \leq x_1 \leq 1} \{ \min \{ -3 + 5x_1, 4 - 6x_1 \} \}.$$

Para encontrar en forma algebraica el valor óptimo de x_1 en la intersección de las dos rectas $-3 + 5x_1$ y $4 - 6x_1$, se establece

$$-3 + 5x_1 = 4 - 6x_1,$$

de donde se obtiene $x_1 = \frac{7}{11}$. Así, $(x_1, x_2) = (\frac{7}{11}, \frac{4}{11})$ es la *estrategia mixta óptima* del jugador 1, y

$$\underline{v} = v = -3 + 5\left(\frac{7}{11}\right) = \frac{2}{11}$$

es el valor del juego.

Para encontrar la estrategia mixta óptima correspondiente del jugador 2, el razonamiento es el siguiente. De acuerdo con la definición del valor minimax $\bar{v}$ y el teorema minimax, el pago esperado que se obtiene con esta estrategia $(y_1, y_2, y_3) = (y_1^*, y_2^*, y_3^*)$ tendrá que satisfacer la condición

$$y_1^*(5 - 5x_1) + y_2^*(4 - 6x_1) + y_3^*(-3 + 5x_1) \leq \bar{v} = v = \frac{2}{11}$$

para todos los valores de x_1 ($0 \leq x_1 \leq 1$). Más aún, cuando el jugador 1 juega de manera óptima —esto es, $x_1 = \frac{7}{11}$ —, esta desigualdad será una igualdad —por el teorema minimax—, de manera que

$$\frac{20}{11}y_1^* + \frac{2}{11}y_2^* + \frac{2}{11}y_3^* = v = \frac{2}{11}.$$

Como (y_1, y_2, y_3) es una distribución de probabilidad, también se sabe que

$$y_1^* + y_2^* + y_3^* = 1.$$

Por lo tanto, $y_1^* = 0$, puesto que $y_1^* > 0$ violaría la penúltima ecuación; es decir, en la gráfica, el pago esperado en el punto $x_1 = \frac{7}{11}$ estaría por encima del punto maximin. (En general, a cualquier recta que no pasa por el punto maximin se le debe asignar un peso de cero para evitar que el pago esperado tenga un valor más alto que este punto.)

Entonces,

$$y_2^*(4 - 6x_1) + y_3^*(-3 + 5x_1) \begin{cases} \leq \frac{2}{11} & \text{para } 0 \leq x_1 \leq 1, \\ = \frac{2}{11} & \text{para } x_1 = \frac{7}{11}. \end{cases}$$

Pero y_2^* y y_3^* son números y entonces el lado derecho es la ecuación de una recta, lo cual es un peso ponderado fijo de las dos rectas “inferiores” de la gráfica. Como la ordenada de esta recta debe ser igual a $\frac{2}{11}$ en el punto $x_1 = \frac{7}{11}$ y como nunca debe exceder $\frac{2}{11}$, la recta es horizontal por necesidad. (Esta conclusión siempre es válida a menos que el valor óptimo de x_1 sea 0 o 1, en cuyo caso el jugador 2 también debe usar una sola estrategia pura.) Por tanto,

$$y_2^*(4 - 6x_1) + y_3^*(-3 + 5x_1) = \frac{2}{11}, \quad \text{para } 0 \leq x_1 \leq 1.$$

Entonces, para obtener y_2^* y y_3^* se seleccionan dos valores de x_1 (como 0 y 1) y se resuelven las dos ecuaciones simultáneas obtenidas. Así,

$$4y_2^* - 3y_3^* = \frac{2}{11},$$

$$-2y_2^* + 2y_3^* = \frac{2}{11},$$

de donde $y_2^* = \frac{5}{11}$ y $y_3^* = \frac{6}{11}$. Por tanto, la *estrategia mixta óptima* del jugador 2 es $(y_1, y_2, y_3) = (0, \frac{5}{11}, \frac{6}{11})$.

Si en algún otro problema ocurre que más de dos rectas pasan por el punto maximin, de manera que más de dos valores y_j^* pueden ser mayores que cero, entonces habría muchos empates en el caso de la estrategia mixta óptima del jugador 2. Una de estas estrategias se puede identificar si se igualan en forma arbitraria todas menos dos de estos valores y_j^* a cero y se obtienen las dos restantes como se acaba de describir. En el caso de esas dos y_j^* , las rectas asociadas deben tener una pendiente positiva en un caso y una pendiente negativa en el otro.

Aunque este procedimiento gráfico se ejemplificó para un problema en particular, en esencia el mismo razonamiento se puede usar para resolver cualquier juego con estrategias mixtas que tenga sólo dos estrategias puras no dominadas de uno de los jugadores. En la sección Worked Examples del sitio en internet de este libro se proporciona **otro ejemplo** donde, en este caso, es el jugador 2 quien tiene sólo dos estrategias no dominadas, de forma que el procedimiento de solución gráfica se aplica en un inicio desde el punto de vista de este jugador.

14.5 SOLUCIÓN MEDIANTE PROGRAMACIÓN LINEAL

Cualquier juego de estrategias mixtas se puede resolver en forma muy sencilla si se lo transforma en un problema de programación lineal. Como se verá, esta transformación requiere apenas un poco más que la aplicación del teorema minimax y el uso de la definición de valor maximin $\underline{v}$ y valor minimax $\bar{v}$.

Primero, considere cómo se encuentra la estrategia mixta del jugador 1. Como se indicó en la sección 14.3,

$$\text{Pago esperado para el jugador 1} = \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_i y_j$$

y la estrategia $(x_1, x_2, \dots, x_m)$ es óptima si

$$\sum_{i=1}^m \sum_{j=1}^n p_{ij} x_i y_j \geq \underline{v} = v$$

Para ilustrar este enfoque de programación lineal, considere de nuevo la variación 3 del problema de la campaña política después de eliminar la estrategia dominada 3 del jugador 1 (vea la tabla 14.6). Como existen algunos elementos negativos en la matriz de pagos reducida, no es evidente si el *valor* del juego v es *no negativo* (ocurre que sí lo es). Por el momento, suponga que $v \geq 0$ y proceda sin hacer ninguno de los ajustes mencionados.

Para escribir el modelo de programación lineal del jugador 1 de este ejemplo, observe que p_{ij} en el modelo general es el elemento del renglón i y la columna j de la tabla 14.6, para $i = 1, 2$ y $j = 1, 2, 3$. El modelo que se obtiene es

$$\text{Maximizar } x_3,$$

sujeta a

$$\begin{aligned} 5x_2 - x_3 &\geq 0 \\ -2x_1 + 4x_2 - x_3 &\geq 0 \\ 2x_1 - 3x_2 - x_3 &\geq 0 \\ x_1 + x_2 &= 1 \end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

La aplicación del método simplex a este problema de programación lineal (después de agregar la restricción $x_3 \geq 0$) da $x_1^* = \frac{7}{11}$, $x_2^* = \frac{4}{11}$, $x_3^* = \frac{2}{11}$ como solución óptima. (Vea los problemas 14.5-8 y 14.5-9.) En consecuencia, la estrategia mixta óptima del jugador 1 de acuerdo al criterio minimax es $(x_1, x_2) = (\frac{7}{11}, \frac{4}{11})$ y el valor del juego es $v = x_3^* = \frac{2}{11}$. El método simplex también produce la solución óptima del dual (que se proporciona en seguida) de este problema, ésta es, $y_1^* = 0$, $y_2^* = \frac{5}{11}$, $y_3^* = \frac{6}{11}$, $y_4^* = \frac{2}{11}$, con lo que la estrategia mixta óptima del jugador 2 es $(y_1, y_2, y_3) = (0, \frac{5}{11}, \frac{6}{11})$.

El dual del problema anterior es el modelo de programación lineal del jugador 2 (el que tiene las variables $y_1, y_2, \dots, y_n, y_{n+1}$) que se mostró antes en esta misma sección. (Vea el problema 14.5-7.) Al sustituir los valores de p_{ij} , dados en la tabla 14.6, este modelo es

$$\text{Minimizar } y_4,$$

sujeta a

$$\begin{aligned} -2y_2 + 2y_3 - y_4 &\leq 0 \\ 5y_1 + 4y_2 - 3y_3 - y_4 &\leq 0 \\ y_1 + y_2 + y_3 &= 1 \end{aligned}$$

y

$$y_1 \geq 0, \quad y_2 \geq 0, \quad y_3 \geq 0.$$

Si se aplica el método simplex directamente a este modelo (después de agregar la restricción $y_4 \geq 0$), se obtiene la solución óptima $y_1^* = 0$, $y_2^* = \frac{5}{11}$, $y_3^* = \frac{6}{11}$, $y_4^* = \frac{2}{11}$ (al igual que la solución óptima dual, $x_1^* = \frac{7}{11}$, $x_2^* = \frac{4}{11}$, $x_3^* = \frac{2}{11}$). En consecuencia, la estrategia mixta óptima del jugador 2 es $(y_1, y_2, y_3) = (0, \frac{5}{11}, \frac{6}{11})$ y de nuevo se puede ver que el valor del juego es $v = y_4^* = \frac{2}{11}$.

Como ya se había determinado la estrategia mixta óptima del jugador 2 cuando se resolvió el primer modelo, no fue necesario resolver el segundo. En general, siempre se pueden encontrar las estrategias mixtas óptimas de *ambos* jugadores con sólo elegir uno de los modelos (cualquiera) y usar el método simplex para obtener una solución óptima y una solución óptima dual.

Cuando se aplicó el método simplex a estos dos modelos de programación lineal, se agregó una restricción de no negatividad que suponía que $v \geq 0$. Si este supuesto se violara, ninguno de los dos modelos tendría soluciones factibles, y el método simplex se detendría con rapidez con este mensaje. Para evitar este riesgo se pudo haber agregado una constante positiva, como 3 (el valor absoluto del elemento más negativo), a todos los elementos de la tabla 14.6. Esto habría aumentado en 3 todos los coeficientes de x_1, x_2, y_1, y_2 y y_3 en las restricciones de desigualdad de los dos modelos. (Vea el problema 14.5-2.)

14.6 EXTENSIONES

Aunque en este capítulo se han considerado sólo los juegos de dos personas y suma cero con un número finito de estrategias puras, la teoría de juegos no se limita a este tipo de juegos. En realidad, se han llevado a cabo investigaciones extensas sobre varios tipos de juegos más complicados, que incluyen los que se resumen en esta sección.

La generalización más sencilla es el *juego de dos personas con suma constante*. En este caso, la suma de los pagos de los dos jugadores es una constante fija (positiva o negativa) sin que importe qué combinación de estrategias se seleccione. La única diferencia con un juego de dos personas con suma cero es que, en este caso, la constante debe ser cero. En su lugar, puede surgir una constante diferente de cero debido a que, además de que un jugador gane lo que el otro pierde, los dos jugadores pueden compartir alguna recompensa (si la constante es positiva) o algún costo (si la constante es negativa) por participar en el juego. Agregar esta constante fija no afecta la estrategia que debe elegirse. Por lo tanto, el análisis para determinar las estrategias óptimas es idéntico al descrito en este capítulo para los juegos de dos personas y suma cero.

Una extensión más complicada es el *juego de n -personas*, en el que pueden participar más de dos jugadores. Esta generalización es de particular importancia puesto que, con frecuencia, en muchas situaciones competitivas, se encuentran involucrados más de dos competidores. Esto puede ocurrir, por ejemplo, en la competencia entre empresas de negocios, en la diplomacia internacional, etc. Desafortunadamente, la teoría existente para tales juegos es mucho menos satisfactoria que la de juegos de dos personas.

Otra generalización es el *juego de suma no cero*, en el que la suma de los pagos a los jugadores no tiene que ser 0 (o ninguna otra constante fija). Este caso refleja el hecho de que muchas situaciones de competencia incluyen aspectos no competitivos que contribuyen a la ventaja o desventaja mutua de los jugadores. Por ejemplo, las estrategias de publicidad de compañías que compiten por un mismo mercado pueden afectar no sólo la distribución de ese mercado sino también el tamaño total del mercado que comparte sus productos. Sin embargo, al contrario de un juego de suma constante, el tamaño de la ganancia mutua (o pérdida) para los jugadores depende de la combinación de las estrategias elegidas.

Como es posible la ganancia mutua, los juegos de suma no cero se subdividen en términos del grado en que se permite que los jugadores cooperen. En un extremo se encuentra el *juego no cooperativo*, donde no hay comunicación entre los jugadores anterior al juego. En el otro extremo está el *juego cooperativo* en el que se permite análisis y acuerdos antes del juego. Ejemplos que se podrían formular como juegos cooperativos son las situaciones competitivas que engloban leyes de comercio exterior entre países o los acuerdos que se toman entre empresas y obreros. Cuando existen más de dos jugadores, los juegos cooperativos permiten también que se formen coaliciones entre algunos o todos los jugadores.

Otra extensión más es la que se llama de *juegos infinitos*, donde los jugadores cuentan con un número infinito de estrategias puras. Estos juegos están diseñados para aquellas situaciones en las que la estrategia seleccionada se puede representar mediante una variable de decisión *continua*. Por ejemplo, la variable de decisión puede ser el tiempo en el que se lleva a cabo cierta acción, o la proporción de recursos propios que se asignan a cierta actividad, en una situación de competencia.

Sin embargo, el análisis que se requiere en estas extensiones que están más allá del juego finito de dos personas con suma cero, es relativamente complejo y no se profundizará en él. (Para mayor información al respecto, consulte las referencias seleccionadas 4, 6, 7, 8 y 10.)

■ 14.7 CONCLUSIONES

El problema general de cómo tomar una decisión en un medio competitivo es bastante común e importante. La contribución fundamental de la teoría de juegos es que proporciona un marco conceptual básico para formular y analizar tales problemas en situaciones sencillas. Sin embargo, existe un gran abismo entre lo que la teoría puede manejar y la complejidad de la mayor parte de las situaciones de competencia que surgen en la práctica. Ello significa que, por lo general, las herramientas conceptuales de la teoría de juegos desempeñan un papel complementario cuando se aplican a esas situaciones.

Dada la importancia general del problema, la investigación sobre este tema continúa con algunos éxitos y pretende extender la teoría a casos más complejos.

■ REFERENCIAS SELECCIONADAS

1. Aumann, R. J. y S. Hart (eds.): *Handbook of Game Theory: With Application to Economics*, vols. 1, 2 y 3, North-Holland, Ámsterdam, 1992, 1994, 2002.

2. Brandenburger, A. y H. Stuart: “Biform Games”, en *Management Science*, **53**(4): 537-549, abril de 2007.
3. Chatterjee, K. y W. F. Samuelson (eds.): *Game Theory and Business Applications*, Kluwer Academic Publishers, Boston, 2001.
4. Forgó, F., J. Szép y F. Szidarovsky: *Introduction to the Theory of Games: Concepts, Methods, Applications*, Kluwer Academic Publishers, Boston, 1999.
5. Hohzaki, R.: “A Search Game Taking Account of Attributes of Searching Resources”, en *Naval Research Logistics*, **55**(1): 76-90, febrero de 2008.
6. Mendelson, E.: *Introducing Game Theory and Its Applications*, Chapman and Hall/CRC Press, Boca Raton, FL, 2005.
7. Meyerson, R. B.: *Game Theory: Analysis of Conflict*, Harvard University Press, Cambridge, MA, 1991.
8. Owen, G.: *Game Theory*, 3a. ed., Academic Press, San Diego, 1995.
9. Parthasarathy, T., B. Dutta y A. Sen (eds.): *Game Theoretical Applications to Economics and Operations Research*, Kluwer Academic Publishers, Boston, 1997.
10. Webb, J. N.: *Game Theory: Decisions, Interaction and Evolution*, Springer, Nueva York, 2007.
11. Zhuang, J., y V. M. Bier: “Balancing Terrorism and Natural Disasters-Defensive Strategy with Endogenous Attacker Effort”, en *Operations Research*, **55**(5): 976-991, septiembre-octubre de 2007.

■ AYUDAS DE APRENDIZAJE PARA ESTE CAPÍTULO EN EL SITIO EN INTERNET DE ESTE LIBRO (www.mhhe.com/hillier)

Ejemplos resueltos:

Ejemplos para el capítulo 14.

Archivos para resolver los ejemplos “Ch. 14—Game Theory”:

Archivos de Excel
Archivos de LINGO/LINDO
Archivos de MPL/CPLEX

Glosario del capítulo 14

Vea en el apéndice 1 la documentación del software.

■ PROBLEMAS

Los símbolos a la izquierda de algunos problemas (o de sus incisos) significan lo siguiente:

- C: Use la computadora con cualquier opción de software disponible (o la recomendada por el profesor) para resolver el problema.

Un asterisco en el número del problema indica que al final del libro se da al menos una respuesta parcial.

14.1-1. El sindicato y la administración de una compañía negocian el nuevo contrato colectivo. Por ahora las negociaciones están congeladas, pues la empresa ha hecho una oferta “final” de un aumento salarial de \$1.10 por hora y el sindicato una demanda “final” de un aumento de \$1.60 por hora. Ambas partes han acordado que un árbitro imparcial establezca el aumento en alguna cantidad entre \$1.10 por hora y \$1.60 por hora (inclusive).

El arbitraje ha pedido a cada parte que le presente una propuesta confidencial de un aumento salarial económicamente razonable y justo (redondeado a los diez centavos más cercanos). Por expe-

riencias anteriores, ambas partes saben que por lo general el árbitro acepta la propuesta del lado que cede más en su cifra final. Si ningún lado cambia su cantidad final o si ambos ceden en la misma cantidad, el arbitraje suele establecer una cifra a la mitad (\$1.35 en este caso). Ahora, cada parte necesita determinar qué aumento proponer para obtener un beneficio máximo.

Formule este problema como un juego de dos personas y suma cero.

14.1-2. Dos fabricantes compiten por las ventas de dos líneas de productos distintas pero igualmente redituables. En ambos casos, el volumen de ventas del fabricante 2 es el triple del que logra el fabricante 1. En vista de algunos avances tecnológicos, ambos harán mejoras importantes a los dos productos, pero no están seguros de la estrategia de desarrollo y comercialización que deben seguir.

Si desarrollan al mismo tiempo las mejoras de los dos productos, ningún fabricante podrá tenerlos listos para la venta antes de 12 meses. Una alternativa es llevar a cabo un “programa intensivo” para desarrollar primero uno de los dos productos y tratar de comercializarlo antes de que la competencia lo haga. Si actúa de